The Parmana Handbook

Every capability, explained by reading the actual source.

Generated 2026-10-20

The Parmana Handbook

Every capability this codebase has, no matter how small, explained by reading the actual source rather than trusting other documentation. Written 2026-09-16 and checked against the code that exists today; where it and the code disagree, the code wins and the chapter is fixed.

How each chapter is structured

Every chapter answers the same six questions, in the same order:

1. **What it is.** One paragraph, plain language.
2. **Why it was built.** The real problem, usually quoted from the source's own comments.
3. **How it works.** Traced to real files, usually with line numbers.
4. **How it enables things, with an example.** A real tutorial or test that proves the behavior, or an honest note that none exists yet.
5. **How to validate it yourself.** The exact files to open.
6. **Integration requirements**, where applicable: env vars, config, external dependencies.

Chapter 1: The Trust Model and Domain Model

What it is

Parmana models every request that could result in a real-world action as a chain of six immutable artifacts: Authority, Authorization, Intent, a Decision produced by evaluating Execution Trust Record that binds all of it into one signed, append-only document. Nothing in this chain is ever mutated after creation. to an old one.

Why it was built

`packages/shared/src/domain/business-transaction.ts` states the design directly in its own doc comment: a `BusinessTransaction` "captures the complete upstream trust chain prior to policy evaluation," and the chain itself is drawn as a diagram right in the source:

```
Authority
     ↓
Authorization
     ↓
Intent
     ↓
BusinessTransaction
     ↓
PolicyReference
```

The reasoning is that a policy decision is only meaningful if you can also prove who asked for it, under what authorization, and exactly what they asked for, all as facts that existed before the decision was made and cannot be edited afterward to match whatever the decision turned out to be. `intent.ts` restates the same chain with `Decision` and `Execution` added on the end, and is explicit that "Intent is evaluated by Policy but is never modified by Policy" and that `decision.ts` "does not create authority… does not grant authorization… does not modify intent." Each artifact in the chain has exactly one job and cannot reach backward to change an earlier one.

How it works

The six domain types

All defined in `packages/shared/src/domain/`, one file per type:

`Authority` (`authority.ts`), the entity behind a request. Has an `authorityId`, an `authorityType` (`AuthorityType.USER | ROLE | SERVICE | ORGANIZATION`), a `principalId`, and an `issuedAt` timestamp. Nothing else. There is no field anywhere on this type, or on anything downstream of it, that says "this authority is an AI" or "this authority is a human" in a way policy evaluation can see.

`Authorization` (`authorization.ts`), proof that an `Authority` approved an intended execution, with an `authorityId`, a `purpose` string, `issuedAt`, and an optional `expiresAt`.

`Intent` (`intent.ts`), the actual business action being requested: `action` (e.g. `"TransferFunds"`), `target` (e.g. `"account/12345"`), and free-form `parameters`.

`BusinessTransaction` (`business-transaction.ts`), the aggregate that wraps `Authority`, `Authorization`, `Intent`, a `PolicyReference`, and the runtime `signals` a policy will evaluate. It carries its own lifecycle (`BusinessTransactionStatus`: `RECEIVED → POLICY_EVALUATED → APPROVED | REJECTED → (OVERRIDDEN) → EXECUTING → EXECUTED | FAILED → VERIFIED`). This is the immutable input `RuntimeEngine.execute()` receives.

`Decision` (`decision.ts`), the pure output of evaluating an `Intent`'s signals against a `Policy`: an `outcome` (`APPROVED | REJECTED`), the `matchedRuleId`, `evaluatedRules` (count), and `matchedPath` (the ordered rule-id trace `PolicyEngine` walked). The last three forward does. A `Decision` never carries anything the caller supplied that wasn't independently produced by evaluation itself.

`Execution` (`execution.ts`), what actually happened while processing the transaction: wraps exactly one `Decision`, has its own `ExecutionStatus` (`PROCESSING | COMPLETED | FAILED`) and `ExecutionMode` (`SYNC | ASYNC`), and optionally carries a hash-chained `previousChainHash` / `chainHash` / `chainSignature` linking it to the prior `Execution` for the same transaction (see `ExecutionChainCrypto`, Chapter 3). Verification recomputes each

chained `Execution`'s hash and checks its signature and its link to the one before. An `Execution` with no chain fields is accepted only before the first chained one, as data from before chaining existed; Every `Execution` is also inside the trust record's own hash and signature, so the chain is a second layer, not the only one.

`ExecutionTrustRecord` (`execution-trust-record.ts`), the aggregate root: one `BusinessTransaction`, arrays of `overrides` / `executions` / `verifications` / `receipts` (arrays, not single values, because a long-running or retried transaction can accumulate more than one of each), an optional signed `authorization` (the exact `SignedExecutionAuthorization` the Execution Gateway accepted, see Chapter 7), a `trustRecordHash`, and a `signature`. This is "the authoritative source for replay, verification, audit, and receipt generation," per its own doc comment, and is itself signed as a whole so that any single field being altered after the fact invalidates the record.

PolicyReference: the field that proves which policy content, not just which version

`policy-reference.ts` carries `name`, `version`, `schemaVersion`, and two fields worth understanding closely: `contentHash` and `governanceAnchor`. Both are marked explicitly as "caller-unsettable" in the source comments, a request-supplied `PolicyReference` never carries either; `RuntimeEngine` computes both, after loading the real policy document, and merges them only into the copy embedded in the trust record's `transaction.policy`, never into the transaction as originally submitted. `contentHash` is a sha256 of the canonicalized policy content actually loaded (closing the gap where an in-place edit to an existing `governanceAnchor` records whether that content is traceable to a completed Policy Governance approval (`VERIFIED | NO_APPROVAL_RECORD | SIGNATURE_INVALID | CONTENT_MISMATCH`), resolved unconditionally regardless of whether execution-time enforcement of that fact is turned on (see Chapter 14).

EvidenceAnchor: the explicit "these three things are linked" pointer

`evidence-anchor.ts` is newer and narrower: a single computed `anchorHash` over `{policyContentHash, governanceAnchorStatus, connectorEvidenceHash}`. Its own doc comment is unusually candid about what it is not: "Not a NEW cryptographic guarantee... What this adds is a single, explicitly-named, independently-computed pointer an auditor can check WITHOUT already knowing to reach into `transaction.policy` and `executions[].evidence.attributes.connector` separately and reconstruct the binding themselves." It exists because an audit governance provenance and actual connector evidence were provably linked, and the honest answer at the time was "implicitly, but nothing says so directly."

Caller-type agnosticism

Nowhere in `packages/runtime/src`, `packages/policy/src`, or `packages/execution-gateway/src` does any evaluation or gateway-release logic branch on `AuthorityType`. A search across all three packages for `AuthorityType.` outside test files returns nothing. The only place `credentialHolderType` / `AuthorityType` is read for a real decision is at the HTTP caller-auth layer, specifically `isHumanCaller()` for Policy Governance approve/reject (Chapter 14), a narrow, deliberate exception, not evidence that the core trust chain treats callers differently. An AI agent, a human, a service account, and an organization-level credential all produce identical `Authority` / `Intent` / `Decision` / `Execution` shapes and go through the exact same `RuntimeEngine.execute()` path.

How it enables things, with a concrete example

`examples/tutorials/100-authorization-caller-type-agnostic` and `packages/api/tests/integration/authority-type-agnostic-execution.integration.test.ts` both exist specifically to prove this: the integration test's own title is "produces an identical APPROVE decision regardless of authority.authorityType, including a value outside the AuthorityType enum entirely," and a second case proves an identical REJECT decision (same `matchedRuleId`, same reason shape) across authority types too. This is the concrete evidence behind the "caller-type-agnostic" claim above, not just an absence-of-special-casing argument.

More broadly, the domain model is what every other capability in this book builds on: Chapter 2's `RuntimeEngine.execute()` walkthrough operates entirely in terms of these types; Chapter 5's runtime pipeline is the process that turns a `BusinessTransaction` into an `ExecutionTrustRecord`; Chapter 6's cryptography chapter explains exactly how `trustRecordHash` / `signature` get computed over this aggregate.

How to validate this yourself

- `packages/shared/src/domain/business-transaction.ts`, `authority.ts`, `authorization.ts`, `intent.ts`, `decision.ts`, `execution.ts`, `execution-trust-record.ts`, `policy-reference.ts`, `evidence-anchor.ts`, the type definitions themselves, each with a doc comment explaining its own role in the chain.

- `packages/api/tests/integration/authority-type-agnostic-execution.integration.test.ts`, proves caller-type agnosticism at the HTTP boundary.
- `examples/tutorials/100-authorization-caller-type-agnostic/run.ts`, the same proof, as a runnable narrative.
- `grep -rn "AuthorityType\." packages/runtime/src packages/policy/src packages/execution-gateway/src` (excluding test files), reproduces the "nowhere is this special-cased" claim directly.

Integration requirements

None specific to the domain model itself, these are plain TypeScript interfaces with no external dependency. What consumes them (the runtime pipeline, storage, crypto) each carries its own requirements, covered in later chapters. The one practical requirement worth naming here: any caller submitting a `BusinessTransaction` must supply a well-formed `Intent` (`action`, `target`, `parameters`) and a `PolicyReference` naming a real, loadable policy, `contentHash` and `governanceAnchor` must never be supplied by the caller, since the server computes and overwrites both regardless of what's submitted.

Chapter 2: Configuration and Bootstrapping

What it is

Every runtime-configurable choice Parmana makes, which storage backend, which signature algorithm, where keys live, whether caller authentication is on, flows through exactly one function: `loadConfig()` in `packages/shared/src/config/Config.ts`. Everything downstream, from which `Signer` gets constructed to whether the process refuses to start at all, is derived from the single immutable object that function returns.

Why it was built

`Config.ts`'s own doc comment states the goal plainly: "Centralized immutable configuration. This is the only configuration model used by Parmana." The practical reason is fail-closed startup. A process that boots successfully, passes `/health`, and only fails on its first real request (because a key file is missing, or a database is unreachable) is worse than a process that refuses to start at all. Several files across `packages/api/src/bootstrap/` exist specifically to convert what would otherwise be a lazy, request-time failure into an eager, startup-time one, and their own comments say so directly: `assertSigningKeyMaterialConfigured.ts` opens with "Without this, `FileKeyProvider` only throws lazily... A production process could otherwise boot 'successfully', pass `/health`, and only fail on its very first real request."

How it works

`loadConfig()`, section by section

`loadConfig()` (`Config.ts:305-381`) reads `process.env` exactly once per call and returns a frozen `Config` object with eleven sections: `environment`, `storage`, `crypto`, `keys`, `secrets`, `authorization`, `policy`, `trust`, `api`, `auth`, `rateLimit`, `logging`. Every value is parsed and validated through `ConfigValidation.ts`'s `parse*` functions, each of which throws immediately on an unrecognized value rather than silently falling back to a default. Two sections fail closed even harder, by refusing an *unset* value rather than just an invalid one: `requirePolicyDirectory()` throws if `PARMANA_POLICY_DIR` is unset, and `assertSigningKeyMaterialConfigured.ts` (a separate, startup-only check, not inside `loadConfig()` itself) does the same for `PARMANA_KEY_DIR`.

`.env` resolution is location-independent: `findEnvFile()` walks up from `Config.ts`'s own directory until it finds a `.env` file, so every package in the monorepo shares the same configuration regardless of which package's `cwd` a script happens to run from.

The full environment variable reference

Every variable below is read somewhere in this monorepo; `.env.example` is the canonical list (351 lines, heavily commented) and was the primary source for this table, cross-checked against `Config.ts` / `ConfigValidation.ts` for the ones that carry a real enum.

Variable	Required?	Valid values	Default	What it contro
`NODE_ENV`	No	free-form; only `"test"` changes behavior anywhere	`development`	Test-mode bypasses (mer storage, skippe startup asserts
`PARMANA_POLICY_DIR`	**Yes**	any directory path	,	Where `policy.json` live
`PARMANA_KEY_DIR`	Yes unless `NODE_ENV=test`	any directory path	,	Where signing `.pem` files live
`PARMANA_KEY_MATERIAL_JSON`	No	JSON object, see below	unset	Materializes ke files from env (platforms with volume/secret- primitive
`PARMANA_GATEWAY_KEY_ID`	No	any string	`gateway`	Key-file prefix the Gateway's

Variable	Required?	Valid values	Default	What it contro
				attestation key
`PARMANA_GATEWAY_ID`	No	`[A-Za-z0-9._-]+`	`parmana-gateway`	Logical Gatewa identity name i audit events
`PORT`	No	number	`3000`	HTTP bind por
`HOST`	No	any	`0.0.0.0`	HTTP bind inte
`SHUTDOWN_TIMEOUT_MS`	No	number	`10000`	Graceful-shutc drain window
`LOG_LEVEL`	No	free-form	`info`	Log verbosity s (no enforcing l library)
`PARMANA_STORAGE`	No	`memory | postgres | supabase`	`memory`	Storage backe business transactions/tr records
`DATABASE_URL`	Yes unless `NODE_ENV=test`	Postgres connection string	,	Direct `pg` connection, us regardless of `PARMANA_STO` by nonce/audit stores
`SUPABASE_URL`	Yes if `PARMANA_STORAGE=supabase`	URL	,	Supabase proj URL
`SUPABASE_SERVICE_ROLE_KEY`	Yes if `PARMANA_STORAGE=supabase`	`sb_secret_*`	,	Privileged serv role key
`SUPABASE_ANON_KEY`	Yes if `PARMANA_STORAGE=supabase`	`sb_publishable_*`	,	Low-privilege a key (used by `scripts/ver policy-chang approved.ts` Chapter 14)
`CRYPTO_MODE`	No	`single | hybrid | pq`	`single`	Whether a secondary signature algor also signs ever trust record/re
`PRIMARY_SIGNATURE_PROVIDER`	No	`ed25519 | ecdsa-p256 | dilithium3 | dilithium5 | sphincs-plus` (also accepts `ml-dsa-65` as an alias for `dilithium3`)	`ed25519`	Primary signatu algorithm
`SECONDARY_SIGNATURE_PROVIDER`	Yes if `CRYPTO_MODE=hybrid`	same list as above	unset	Secondary signature algor must differ fro primary
`HYBRID_SIGNATURE_REQUIRED`	No	`"true"` (strict equality)	`false`	Rejects a recor whose `signatures` is absent/parti once hybrid m trusted
`HASH_PROVIDER`	No	`sha256 | sha3-512 | blake3`	`sha256`	Hash algorithm
`KEY_PROVIDER`	No	`local | aws-kms | azure-key-vault | gcp-kms | hsm`	`local`	Signing-key custody backe see the note be `local` and a `kms` are real; t

Variable	Required?	Valid values	Default	What it contro
				other three are reserved
`AWS_REGION`	Yes if `KEY_PROVIDER=aws-kms`	AWS region string	,	Region for the `KMSClient`
`AWS_ROLE_ARN`	No	AWS role ARN	unset	If set, exchang Vercel's OIDC for short-lived credentials ins of the SDK's d credential chai
`EXECUTION_AUTHORIZATION_TTL_SECONDS`	No	number	`120`	Default TTL on signed Executi Authorization
`TRUST_PROFILE`	No	`v1` (only value defined)	`v1`	Trust record schema profile
`RECEIPT_VERSION`	No	free-form	`1`	Receipt schem version string
`PARMANA_API_KEYS`	Yes unless `PARMANA_AUTH_DISABLED=true`	JSON array, see Chapter 13	`[]`	Caller identities their key hashe
`PARMANA_AUTH_DISABLED`	No	`"true"` (strict equality)	`false`	Bypasses calle authentication entirely, local development o
`RATE_LIMIT_EXECUTE_PER_MINUTE`	No	number	`30`	`/execute`, ke by caller identi
`RATE_LIMIT_HEALTH_PER_MINUTE`	No	number	`300`	`/health`, `/ready`, keye IP
`HUBSPOT_PRIVATE_APP_TOKEN`	No	HubSpot token	unset	HubSpot conne connector simp isn't registered unset
`GITHUB_APP_ID` / `GITHUB_INSTALLATION_ID` / `GITHUB_APP_PRIVATE_KEY`	No, but all-or-nothing	,	unset	GitHub connec
`PAYTM_CONNECTOR_URL` / `PAYTM_CONNECTOR_SHARED_SECRET`	No, but all-or-nothing	HTTPS URL / string	unset	Paytm connect (forwards to a separate truste service, never Paytm directly)
`PARMANA_SECRETS_PROVIDER`	No	`env | aws-secrets-manager`	`env`	Where connec *bearer* credent resolve from
`DATABASE_PROVIDER`	**Never set**	,	,	Retired; setting throws at start naming `PARMANA_STO` as the replacer

A genuine discrepancy worth flagging directly: `.env.example`'s own comment on `KEY_PROVIDER` says "Only `local` (FileKeyProvider) has an implementing class today, setting this to any of the other four values does nothing." That statement is now wrong. `SignerBootstrap.ts` (Chapter 3) implements `aws-kms` for real, via `KmsSigner`. The comment predates that work and was not updated afterward, exactly the kind of drift this book exists to catch rather than repeat.

`*Bootstrap` classes: composition roots, not dependency injection

Three classes in `packages/crypto/src/` follow an identical shape: a static `create()` method, a private static cache, construction driven entirely by `loadConfig()`.

- **`CryptoBootstrap`** (`CryptoBootstrap.ts`) builds a `CryptoProvider` (hash + signature) for a given algorithm, registering built-in providers (`SHA256HashProvider`, `Ed25519SignatureProvider`, `Dilithium3SignatureProvider`) into `HashRegistry` / `SignatureRegistry` and selecting by `config.crypto.hashProvider` / `primarySignatureProvider`. `createHybrid()` builds both primary and secondary providers at once for `CRYPTO_MODE=hybrid`.
- **`KeyBootstrap`** (`KeyBootstrap.ts`) is the older `KeyProvider` composition root. Its own doc comment is unusually direct about its current status: `KEY_PROVIDER` accepts five values for forward compatibility, but "only `FileKeyProvider` (`local`) is actually implemented," and it throws loudly for anything else rather than silently falling back to file-based keys, closing a real prior gap where a misconfigured `aws-kms` value used to parse cleanly and then quietly construct a `FileKeyProvider` anyway.
- **`SignerBootstrap`** (`SignerBootstrap.ts`) is the newer, signing-capable sibling (Chapter 3 covers `Signer` vs `KeyProvider` in full). Unlike `KeyBootstrap`, it supports both `local` (`LocalFileSigner`) and `aws-kms` (`KmsSigner`) for real, and is deliberately **not** memoized the way `KeyBootstrap.create()` is, a static singleton here previously poisoned test isolation, since each test sets its own `PARMANA_KEY_DIR` and a cached signer would keep pointing at a deleted temp directory across tests.

Startup order, traced from `server.ts`

`packages/api/src/server.ts` runs, in this exact order:

1. `assertStorageConfigured()`, refuses to start with `PARMANA_STORAGE=supabase` and no `DATABASE_URL`.
2. `assertSigningKeyMaterialConfigured()`, materializes `PARMANA_KEY_MATERIAL_JSON` into `PARMANA_KEY_DIR` for any file not already present, then confirms `default.private.pem` / `default.public.pem` exist (skipped for `KEY_PROVIDER=aws-kms`, which has no local file for that key by design).
3. `await assertKmsSigningKeyReachable()`, for `KEY_PROVIDER=aws-kms` only: constructs a real `SignerBootstrap`-backed signer and confirms the configured KMS key actually exists and is reachable, before binding the port.
4. `assertPaytmConnectorConfigured()`, refuses to start with only one of `PAYTM_CONNECTOR_URL` / `PAYTM_CONNECTOR_SHARED_SECRET` set.
5. `createExecutionSystem()`, `createApplication()`, `createCallerAuthenticator()`, Chapter 12), the actual application graph.
6. `assertPolicyGovernanceIntegrityAtStartup()`, awaited before the port binds. It refuses to start if an approved policy's live file does not match its approval record, or if the approval records cannot be read (Chapter 7 and Chapter 14).
7. `app.listen(PORT, HOST)`, the port only binds after every check above has passed.
8. `schedulePolicyGovernanceIntegrityCheck()`, fired *after* the port is bound, deliberately fail-open (Chapter 14). It re-runs the integrity check on a timer and logs. These must never delay traffic from being accepted, unlike steps 1-6.
9. `createGracefulShutdown()` wired to `SIGTERM` / `SIGINT`.

The split between steps 1-6 (fail-closed, before the port binds) and step 8 (fail-open, after) is a deliberate, named discipline, not an accident of ordering, durable storage and signing key material are things a process cannot function without at all, while a Policy Governance integrity mismatch is something to detect and log, not something that should keep a healthy process from serving traffic.

How it enables things, with a concrete example

Every tutorial in `examples/tutorials/` that spins up a real `createApplication()` instance (the large majority of them) depends on `loadConfig()` succeeding first, `examples/tutorials/89-readiness-probe` specifically exercises the boundary between "storage is configured but unreachable" (`NOT_READY`, 503) and "storage isn't configured to need reaching at all" (`READY`, memory-backed), which is this exact configuration model in action. `examples/tutorials/113-kms-key-id-resolution` exercises `resolveKmsKeyId()`, one concrete function this configuration ultimately drives (Chapter 3 covers it in depth).

How to validate this yourself

- `packages/shared/src/config/Config.ts`, `ConfigValidation.ts`, `StorageProviders.ts`, `KeyProviders.ts`, `SecretsProviders.ts`, `CryptoAlgorithms.ts`, `TrustProfiles.ts`, the full config model and its validation.
- `packages/api/src/bootstrap/assertStorageConfigured.ts`, `assertSigningKeyMaterialConfigured.ts`, `assertPaytmConnectorConfigured.ts`, the fail-closed startup checks, each with a doc comment explaining exactly what it prevents.
- `packages/api/src/server.ts`, the real, ordered bootstrap sequence.

- `packages/crypto/src/CryptoBootstrap.ts`, `KeyBootstrap.ts`, `SignerBootstrap.ts`, the three composition roots.
- `.env.example`, the canonical, heavily-commented variable reference (verify against `ConfigValidation.ts` for anything you're not sure is still accurate, per the `KEY_PROVIDER` drift noted above).

Integration requirements

A minimal working `.env` needs, at least: `PARMANA_POLICY_DIR`, `PARMANA_KEY_DIR` (with a real `default.private.pem` / `default.public.pem` pair present, or `PARMANA_KEY_MATERIAL_JSON` set), and either `PARMANA_STORAGE=memory` (no further storage config needed) or `PARMANA_STORAGE=supabase` plus `DATABASE_URL` / `SUPABASE_URL` / `SUPABASE_SERVICE_ROLE_KEY` / `SUPABASE_ANON_KEY`. Caller authentication needs either `PARMANA_API_KEYS` populated or `PARMANA_AUTH_DISABLED=true` (development only). `KEY_PROVIDER=aws-kms` additionally needs `AWS_REGION` and either `AWS_ROLE_ARN` (Vercel OIDC federation) or a real AWS credential chain reachable in the runtime environment (`~/.aws/credentials` locally, an instance/task role elsewhere).

Chapter 3: Cryptography

What it is

`packages/crypto` is where every signature Parmana produces or checks actually happens: signing an `ExecutionTrustRecord`, an `ExecutionAuthorization`, a `Receipt`, a `PolicyChangeApprovalRecord`, a caller-audit chain entry, and verifying every one of those independently. It provides a pluggable set of hash and signature algorithms, a deterministic serialization step every one of them depends on, and two key-custody models: local files on disk, and AWS KMS ("sign-without-release," the private key never leaves KMS).

Why it was built

The trust model in Chapter 1 is only as strong as the signatures backing it. Every artifact in the chain of custody claims to be "signed by Parmana," and that claim needs to be checkable by a third party who was never in the room, using nothing but the public key and the artifact itself. That requirement drives three design choices covered below: canonical serialization (so the same logical content always produces the same bytes to sign, regardless of how it was constructed in memory), a `Signer` abstraction separate from `KeyProvider` (so a key that can never be exported, like an AWS KMS key, is a first-class option, not a workaround), and hybrid signing (so a future break of one algorithm doesn't retroactively invalidate everything signed under it, as long as a second algorithm was also applied).

How it works

`CanonicalSerializer`: the thing every signature is actually computed over

`packages/crypto/src/CanonicalSerializer.ts` is nineteen lines of `normalize()` plus a `serialize()` that runs `JSON.stringify` over the result. The normalization rule: objects have their keys recursively sorted lexicographically before serialization; arrays keep their original order (element order is meaningful, key order in an object is not); `Date` becomes its ISO string; everything else passes through unchanged. This single function is what makes two logically-identical objects with different key insertion order, or objects that came from different sources (a hand-typed JSON literal vs. a value round-tripped through a Postgres JSONB column, which does not guarantee to preserve key order) hash and sign identically. Every cryptographic operation in this codebase is required to go through it, `ArtifactSigner.sign()`, `TrustRecordHasher.hash()`, and every `*Crypto` class (`PolicyChangeCrypto`, `RefusalCrypto`, `VerificationCrypto`, `AuditEventCrypto`) all construct their own `CanonicalSerializer` instance rather than hashing raw `JSON.stringify` output directly.

2026-09-16, a batch of policy approvals initially appeared to fail a content-hash check purely because a database round-trip had reordered object keys in the stored `proposedContent`, that turned out to be a red herring, since `CanonicalSerializer` normalizes object key order away. The real, separate finding underneath it was that the stored content was missing an entire field (`unboundSignalReasons`, and for two policies, `boundSignals`) relative to the current file, a genuine content difference no amount of key-order normalization could paper over. See §2.26's "Legacy-policy backfill" entry for the full account, worth reading as a caution against assuming a hash mismatch has an obvious cause before actually diffing the two objects field by field.

Hash and signature providers: pluggable, registered, resolved through `CryptoBootstrap`

`packages/crypto/src/providers/hash/SHA256HashProvider.ts` implements `HashProvider`: one method, `hash(data: Uint8Array): Promise<string>`, returning a hex digest via Node's `node:crypto` `createHash("sha256")`. Two more hash algorithms (`sha3-512`, `blake3`) are recognized by `Config.ts`'s validation but have no registered provider class yet, selecting either at the config layer would fail when `HashRegistry.get()` tries to resolve them; `sha256` is the only one with a real implementation.

Signature providers implement `SignatureProvider` (`sign` / `verify`, plus a readonly `algorithm` field): `Ed25519SignatureProvider.ts` (46 lines, wraps `node:crypto`'s `sign` / `verify` with no options object, i.e. pure Ed25519, not Ed25519ph) and `Dilithium3SignatureProvider.ts` (53 lines, near-identical shape, using Node's native `"ml-dsa-65"` key type, this is the ML-DSA-65/FIPS 204 standard, and `ConfigValidation.ts` accepts the string `"ml-dsa-65"` as a config-time alias that resolves to the same internal `"dilithium3"` identifier, so an operator can write either name in `PRIMARY_SIGNATURE_PROVIDER`). ML-DSA-65 signatures are randomized, signing the same message twice with the same key produces two different, both valid, signatures, noted directly in that provider's own doc comment, worth knowing before assuming a signature mismatch across two runs means something is

broken. `MlDsaSupport.ts` detects at runtime (cached after first call) whether the current Node/OpenSSL build actually supports `generateKeyPairSync("ml-dsa-65")`, this requires Node ≥24 with OpenSSL ≥3.5; tests that need it skip cleanly with `ML_DSA_65_SKIP_REASON` on older runtimes rather than failing. Two more algorithms (`ecdsa-p256`, `sphincs-plus`) are recognized by config validation with no registered provider at all.

`CryptoBootstrap.create()` is the actual resolution point: it builds a `CryptoProvider` (hash + signature bundled) by registering the built-in providers into `HashRegistry` / `SignatureRegistry` and selecting by whatever `loadConfig().crypto.hashProvider` / `primarySignatureProvider` say. `createHybrid()` builds both primary and secondary providers at once, throwing if `CRYPTO_MODE=hybrid` but no `SECONDARY_SIGNATURE_PROVIDER` is set. `ProviderFactory` (`providers/ProviderFactory.ts`) is a three-line compatibility wrapper around `CryptoBootstrap.create()`, nothing more.

`Signer` vs. `KeyProvider`: two abstractions for a real constraint

`KeyProvider` (`KeyProvider.ts`) has a `getPrivateKey(keyId): Promise<KeyObject>` method, it assumes the caller can obtain the raw private key material. That assumption is structurally false for AWS KMS, an HSM, or Vault Transit, where the entire point is that the private key never leaves the custody boundary. `Signer` (`Signer.ts`) exists for exactly that case: it drops `getPrivateKey` and adds `sign(keyId, data): Promise<string>` instead, the backend signs on the caller's behalf and only ever returns a signature, never key material. Read operations (`getPublicKey`, `getMetadata`, `hasKey`, `listKeys`) are identical on both interfaces, since verification never needed private key material regardless of custody model.

Two implementations exist for each side today. `FileKeyProvider` (`providers/key/FileKeyProvider.ts`, 192 lines) reads `<keyId>.private.pem` / `<keyId>.public.pem` from `PARMANA_KEY_DIR`, this is the only real `KeyProvider`, and `KeyBootstrap.create()` (Chapter 2) throws for any other `KEY_PROVIDER` value rather than silently falling back to it. `LocalFileSigner` wraps that same file-reading logic behind the `Signer` interface (so `local` custody works through either abstraction), and `KmsSigner` (`providers/signer/KmsSigner.ts`, 217 lines) is the real AWS KMS implementation, resolved only through `SignerBootstrap`, never through `KeyBootstrap`.

`KmsSigner`: sign-without-release, against real AWS KMS

`KmsSigner` supports exactly one key spec/algorithm pair, `ECC_NIST_EDWARDS25519` / `ED25519_SHA_512`, matching this codebase's `ed25519` default, noted in-source as deliberate: AWS KMS added Ed25519 support in November 2025, so adopting it needed no signature-algorithm migration. `sign()` / `getPublicKey()` / `getMetadata()` / `hasKey()` each call a real `KMSClient` command (`SignCommand`, `GetPublicKeyCommand`, `DescribeKeyCommand`); `listKeys()` returns only the logical ids this process signs with (`default` and the current `PARMANA_VERIFICATION_KEY_ID`), each if its alias exists, so `GET /.well-known/jwks.json` answers under KMS. It does not enumerate the account's keys: an account wide list would publish keys unrelated to Parmana. (Before 2026-10-03 it was unimplemented, and the endpoint answered `501` in production.)

Credentials are never a static access key/secret pair read from this codebase's own configuration. If `AWS_ROLE_ARN` is set, `KmsSigner` dynamically imports the optional `@vercel/oidc-aws-credentials-provider` peer dependency and exchanges Vercel's per-invocation OIDC token for short-lived STS credentials; otherwise it falls back to the AWS SDK's own default credential provider chain.

`resolveKmsKeyId()` is a small, standalone, exported function (`KmsSigner.ts`) that matters more than its size suggests. Every real signing call site in this codebase passes a *logical* keyId, `"default"`, or a tenant-scoped `"tenant.acme"`, never a raw AWS identifier. AWS KMS's `KeyId` parameter requires a real key ID (UUID), a full ARN, or an alias name/ARN (which must carry the `alias/` prefix); a bare `"default"` matches none of those and AWS rejects it outright. `resolveKmsKeyId()` maps a bare logical keyId to `alias/<keyId>` (mirroring `FileKeyProvider`'s own `<keyId>.private.pem` filename convention), with each `.` replaced by `/` because KMS alias names cannot contain a dot, so `"tenant.acme"` is `alias/tenant/acme`. It passes an explicit `alias/` name through unchanged, and refuses AWS's reserved `alias/aws/` prefix. It refuses a full key ARN: a keyId often comes from the record being verified, and an ARN can name a key in another AWS account whose public key anyone may read (audit, 2026-10-03). A UUID shaped keyId is treated as an alias name, never as a raw key ID. A class that passes the logical keyId straight through would call AWS with `{ KeyId: "default" }` and fail on every signing attempt. Because of this function, `alias/default` has to actually exist in AWS, it is not a naming convenience, it is the literal resolution target for the logical id every call site already uses.

Large messages: the 4096 byte KMS limit and the commitment scheme

AWS KMS refuses a raw Ed25519 message longer than 4096 bytes. `KmsSigner.sign()` originally passed the full canonical bytes of whatever it was signing, so signing a full `ExecutionTrustRecord` (with its bound authorization,

connector evidence and governance anchor) failed with `ValidationException: Member must have length less than or equal to 4096`. because earlier KMS tests only used the small `test:fixture-execute` record.

The fix is `packages/crypto/src/SignatureCommitment.ts`. A message of 4096 bytes or fewer is signed raw, exactly as before. A longer message is signed as a fixed 97 byte commitment: the prefix `PARMANA-ED25519-LARGE-MESSAGE-V1`, a NUL byte, and the SHA-512 digest of the message. The signature is still ordinary Ed25519. The scheme depends only on message length, so nothing is stored on a record to say which form was used, and every signature issued before the change verifies unchanged.

Verification lives in `Ed25519SignatureProvider.verify()`, which every TypeScript verification path goes through. It tries the raw signature first, and only for a message over 4096 bytes also tries the commitment form. A commitment signature over a small message is rejected, so a small message cannot be downgraded to the commitment form. The prefix contains a NUL byte, which a canonical JSON message never starts with, so the two forms cannot be confused.

Two consequences worth knowing. A verifier written elsewhere has to implement the same rule to verify a large record signed under KMS, and the Python SDK's offline verifier already does. This also does not change the order of operations in a request: the connector is still called before the Execution Trust Record is signed. released action always has signed evidence behind it (Chapter 8).

`SignerKeyProviderAdapter`: closing a real signing/verification divergence

every authorization it verifies, including ones signed under the plain `"default"` keyId, not only tenant-scoped ones, contrary to an earlier assessment in this same codebase's own comments that this path was "currently inert." `createExecutionGateway.ts` unconditionally constructed a fresh `new FileKeyProvider()` for that lookup, regardless of `KEY_PROVIDER`. Once `KEY_PROVIDER=aws-kms` was set, every authorization was *signed* by the real KMS key (via to still be materialized from a pre-migration `PARMANA_KEY_MATERIAL_JSON` entry. Signing and verification silently used two different keys, every real request's `signatureVerified` check (and everything that cascades from it: `businessTransactionHashMatches`, `nonceUnseen`) failed. `SignerKeyProviderAdapter` closes this by wrapping the *same* `Signer` instance `createGatewayPublicKey()` already uses, so signing and per-authorization verification now resolve through one identical source, KMS-backed or file-backed, whichever `KEY_PROVIDER` actually says.

`HybridSignatureProvider`: two independent signatures, fail-closed on either

`HybridSignatureProvider.ts` is not a `SignatureProvider` implementer, that interface signs with exactly one key and produces exactly one signature. Hybrid mode needs two of each. `sign()` produces a fixed `[primary, secondary]` pair of `SignatureEntry` values; `verify()` requires *exactly* two entries, one matching each configured algorithm, both independently verified, a missing, extra, duplicated, or mismatched-algorithm entry is rejected outright, never a partial pass. The stated purpose (its own doc comment) is "harvest now, decrypt/forge later" defense for the classical-to-post-quantum transition: if a future quantum computer ever breaks the classical primary algorithm, the post-quantum secondary signature alone still holds, and vice versa if a weakness is ever found in the newer PQ scheme instead. `HYBRID_SIGNATURE_REQUIRED=true` (Chapter 2) makes `VerificationCrypto` reject any record whose `signatures` array is absent or partial, rather than silently falling back to the legacy single-signature check, off by default so turning `CRYPTO_MODE=hybrid` on never retroactively invalidates records signed before that flag was set.

A note on what `.env.example` gets wrong

`.env.example`'s comment on `KEY_PROVIDER` states that only `local` has an implementing class and that setting it to anything else "does nothing." That was true when written and is no longer true: `SignerBootstrap.ts` implements `aws-kms` for real. Separately, a live `.env` in this repository was observed carrying `KMS_REGION` and `KMS_KEY_ALIAS` variables, neither name is read anywhere in `packages/*/src` (confirmed by a repo-wide grep returning zero matches). The variables `KmsSigner` / `assertKmsSigningKeyReachable` actually read are `AWS_REGION` and `AWS_ROLE_ARN`. If you are configuring KMS custody for a real deployment, set those two, not `KMS_REGION` / `KMS_KEY_ALIAS`, the latter pair currently does nothing at all.

How it enables things, with a concrete example

- `examples/tutorials/113-kms-key-id-resolution` exercises `resolveKmsKeyId()` directly against six real input shapes (bare logical id, tenant-scoped id, already-qualified alias, full ARN, reserved `alias/aws/` name, raw UUID), no AWS credentials needed since it's a pure string-mapping function.
- `examples/tutorials/114-signing-verification-key-agreement` exercises the `SignerKeyProviderAdapter` fix, proving signing and verification now resolve through the same key source.

- `examples/tutorials/47-canonical-json` demonstrates `CanonicalSerializer`'s normalization behavior directly.
- `packages/crypto/tests/unit/kms-signer.test.ts` and `signer-key-provider-adapter.test.ts` migration's own troubleshooting guide notes a real, one-time end-to-end test was also run against actual AWS KMS before this coverage was trusted, then deleted once confirmed.

How to validate this yourself

- `packages/crypto/src/CanonicalSerializer.ts`, read the whole file, it's short and the normalization rule is the single most load-bearing piece of logic in this chapter.
- `packages/crypto/src/CryptoBootstrap.ts`, `KeyBootstrap.ts`, `SignerBootstrap.ts`, the three composition roots (Chapter 2 also covers these from the config-and-startup angle).
- `packages/crypto/src/Signer.ts`, `KeyProvider.ts`, the two interfaces, both with doc comments explaining exactly why they're separate.
- `packages/crypto/src/providers/signer/KmsSigner.ts`, `providers/SignerKeyProviderAdapter.ts`,

Integration requirements

For local-file signing (the default): `PARMANA_KEY_DIR` pointing at a directory containing `default.private.pem` / `default.public.pem` (Ed25519 PEM pair), or `PARMANA_KEY_MATERIAL_JSON` set so the process materializes them itself at startup.

For AWS KMS signing: `KEY_PROVIDER=aws-kms`, `AWS_REGION` set, an Ed25519 KMS key (`ECC_NIST_EDWARDS25519` key spec) with an alias matching your logical keyId (`alias/default` for the default signing key), and either `AWS_ROLE_ARN` (for Vercel OIDC federation) or a real AWS credential chain reachable in the runtime environment. `@vercel/oidc-aws-credentials-provider` is an optional peer dependency, only needed if `AWS_ROLE_ARN` is set. Note the Vercel-specific cold-start constraint from the troubleshooting guide's item 3: on Vercel specifically, KMS calls that need the OIDC token cannot run at module top level, they need a real in-flight HTTP request to read the token header from. A long-running process (`server.ts`) is not subject to this, since it calls `assertKmsSigningKeyReachable()` once at real process startup, not per-request.

For hybrid mode: `CRYPTO_MODE=hybrid`, `SECONDARY_SIGNATURE_PROVIDER` set to a different algorithm than `PRIMARY_SIGNATURE_PROVIDER`, and a second key pair present at `<PARMANA_KEY_DIR>/default-secondary.{private,public}.pem`.

Chapter 4: The Policy Engine and Evaluation

What it is

`PolicyEngine` (`packages/policy/src/PolicyEngine.ts`) is the piece of Parmana that turns a policy document and a set of runtime signals into one decision: approve or reject, with a reason and a trace of which rule matched. It is deliberately narrow. Its own doc comment states what it must never do: authorize execution, execute business actions, access external systems, create trust records, replay, or generate timestamps. It is a pure function from `(Policy, PolicySignals)` to `PolicyDecision`.

Why it was built

A policy engine that could reach outside itself (call a database, hit the network, read a clock) would make every decision harder to reason about, replay, or audit. Parmana's design keeps decisioning pure and pushes every side effect (independent verification of signals, persistence, signing) into layers around it. `PolicyEngine` is the load-bearing center that everything else in this chapter's neighboring chapters (signal binding, capability binding, signal-state verification) exists to protect the inputs of.

How it works

The Policy document

A `Policy` (`packages/policy/src/types/Policy.ts:198-300`) has an identity (`policyId`, `policyVersion`, `schemaVersion`), an optional `signalsSchema` declaring each signal's type (`boolean`, `number` or `string`; a request whose signal has another type is refused before any rule runs, as `signal-type-violation`), optional `boundSignals` and `unboundSignalReasons` (covered fully in Chapter 5), and an ordered array of `rules`.

Each `PolicyRule` (`Policy.ts:178-193`) has an `id`, a `condition`, and an `outcome` (`{ action: "approve" | "reject", reason: string }`). Conditions are recursively composable (`Policy.ts:169-173`):

- **Leaf** (`PolicyLeafCondition`): `{ fact, operator, value? }`, a single comparison against one named signal.
- **`all`**: logical AND over child conditions.
- **`any`**: logical OR over child conditions.
- **`always`**: unconditionally true, used as the final catch-all rule.

The full set of supported operators (`PolicyValidator.OPERATORS`, `packages/policy/src/PolicyValidator.ts`):

Category	Operators
Equality	`eq`, `neq`
Numeric	`gt`, `gte`, `lt`, `lte`, `between`
Collection	`in`, `not_in`, `contains`, `not_contains`, `contains_all`, `contains_any`
String	`starts_with`, `ends_with`, `matches`
Existence	`exists`, `not_exists`
Boolean	`is_true`, `is_false`
Null	`is_null`, `is_not_null`
Length	`length_eq`, `length_gt`, `length_gte`, `length_lt`, `length_lte`
Type	`type_is`

The actual comparison logic lives in `OperatorEvaluator` (`packages/policy/src/OperatorEvaluator.ts`, imported by `PolicyEngine.ts:1`), `PolicyEngine` itself only walks the condition tree and delegates each leaf to it.

Evaluation: first-match-wins

`PolicyEngine.evaluate(policy, signals)` (`PolicyEngine.ts:35-55`) calls a private `findFirstMatch`, which iterates `policy.rules` in array order, pushing each rule's `id` onto a `trace` array as it's visited, and returns the first rule whose condition evaluates `true` against the supplied `signals`. This is deterministic by construction: the same

policy and the same signals always produce the same matched rule, because iteration order is fixed array order, not a scored or prioritized search.

A missing fact never satisfies a leaf condition (`PolicyEngine.ts:93-95`: `if (signal === undefined) return false`), an absent signal is not treated as a wildcard or a pass.

If no rule matches, `PolicyDecision.outcome` defaults to `REJECT` via `toOutcome`'s `default` branch (`PolicyEngine.ts:140-142`) with `reason: "no_rule_matched"` and `matchedRuleId: "none"`. Every real policy in this codebase ends with an explicit `always: true` catch-all rule specifically so this default path is never actually reached in practice, it exists as a safety net, not the intended way to reject.

The returned `PolicyDecision` (`packages/policy/src/types/PolicyDecision.ts`) carries `policyId`, `policyVersion`, `outcome`, `reason`, `matchedRuleId`, `evaluatedRules` (how many rules were checked before a match), and `matchedPath` (the full trace of rule IDs visited), enough for an auditor to reconstruct exactly why a decision came out the way it did, without re-running the engine.

Loading and validating: `PolicyRouter`

`PolicyRouter` (`packages/policy/src/PolicyRouter.ts`) is the layer above `PolicyEngine` that actually loads a policy by `(name, version)` via a `PolicyRepository`, then validates it before handing it back. Two validator calls happen on every load:

1. `validator.validate(policy)` (`PolicyRouter.ts:24`), **fail closed**. Throws `PolicyValidationError` on structural problems (missing identity fields, empty `rules`, malformed `boundSignals` / `unboundSignalReasons`, an oversized or dangerous `matches` regex, see below) and, critically, on any rule-referenced fact that is neither in `boundSignals` nor `unboundSignalReasons` (`findUncoveredFacts`, called internally at `PolicyValidator.ts:229`). An uncovered fact is a hard failure, not a warning: every signal a rule can reference must be either bound to the Intent or explicitly acknowledged as independently verified. **Since 2026-09-30** it also refuses any policy with an approve rule that does not require a fact from `approvalSignals` with `is_true`, as its whole condition or directly inside its top level `all` (`validateEveryApprovalNeedsSignedApproval`): no AI agent action is authorized without a signed human approval, reads included (Human approval). The same check runs on every proposed policy change, so such a policy cannot be approved through governance either.
2. `validator.findRuleConflicts(policy)` (`PolicyRouter.ts:30`), **advisory only**. Detects rule pairs whose conditions can both be true, which (under first-match-wins) means the earlier rule silently shadows the later one. Logged via `console.warn` with event `policy_rule_conflict_detected`, never thrown. `PolicyValidator.ts:442-466`'s own doc comment explains why: unlike an uncovered fact (unambiguous fix, bind it or acknowledge rule followed by a broader fallback). The analysis is also incomplete by design, it only reasons about pairs of single, non-nested-fact conditions; anything involving `all` / `any` is reported as `NEEDS_REVIEW` (level `INFO`) without further analysis.

The `matches` regex guard

Because a `matches` condition is evaluated against live, potentially attacker-influenced signal values, `PolicyValidator.validateRegex` (`PolicyValidator.ts:361-391`) rejects two things before a policy is ever accepted: a pattern longer than 200 characters (`MAX_PATTERN_LENGTH`, `PolicyValidator.ts:331`), and a pattern containing a quantified group whose own contents are themselves quantified (e.g. `(a+)+`), the textbook shape of catastrophic regex backtracking (ReDoS). The doc comment is explicit that this is a heuristic, not a proof: it catches the single-level nested-quantifier case, not every pattern capable of exponential-time backtracking. A linear-time engine or an execution timeout would be needed to close that gap completely; this is a deliberate, bounded improvement over no check at all.

Two `PolicyRepository` implementations

`PolicyRepository` (`packages/policy/src/PolicyRepository.ts`) is a three-method interface: `load`, `save`, `listAll`. Two implementations exist:

- `FilePolicyRepository`, reads/writes `policies/{name}/{version}/policy.json` on the local filesystem. Used in local development and in tests.
- `SupabasePolicyRepository` (`packages/policy/src/SupabasePolicyRepository.ts`), reads/writes a `policies` table via a direct Postgres connection. Serverless functions run on a read-only filesystem, so a live policy write to local files cannot succeed. Deployments therefore store policies in Postgres.

`PolicyRegistry` (`packages/policy/src/PolicyRegistry.ts`) is a separate, much smaller piece, an in-memory `Map` from `"name:version"` to registration metadata (`{ name, version, path }`). Its own doc comment is explicit that it

does not load, evaluate, or choose policies; it only tracks what's available.

How it enables things, with a concrete example

- **Tutorial 02** (`examples/tutorials/02-policy-evaluation/run.ts`) exercises `PolicyEngine` directly against a real policy document, showing both the approve and reject paths.
- **Tutorial 04** (`examples/tutorials/04-policy-router/run.ts`) exercises `PolicyRouter` loading a real policy from disk, including its validation step.
- **Tutorial 17** (`examples/tutorials/17-multi-policy-routing/run.ts`) demonstrates routing across multiple distinct policies by name/version.
- **Tutorial 116** (`examples/tutorials/116-supabase-policy-repository/run.ts`, 2026-09-16) demonstrates `SupabasePolicyRepository` directly, including the `EROFS` failure it fixes.

How to validate this yourself

- Read the engine itself: `packages/policy/src/PolicyEngine.ts` (145 lines, worth reading in full).
- Read `packages/policy/src/types/Policy.ts` for the exact shape of every field a policy document can have.
- Run `packages/policy/tests/unit/PolicyEngine.test.ts`, `packages/policy/tests/unit/PolicyValidator.test.ts`, and `packages/policy/tests/unit/PolicyRouter-boundSignals-coverage.test.ts` to see the full assertion suite, including edge cases (missing facts, conflicting rules, oversized regex patterns) not covered above.
- Pick any real policy under `policies/*/*/policy.json` and trace a few signal combinations through its rules by hand, the format is simple enough to do this without running code.

Integration requirements

None beyond what's already required to run the API at all: `PARMANA_POLICY_DIR` (for `FilePolicyRepository`, local/test default `./policies`) or `PARMANA_STORAGE=supabase` plus `DATABASE_URL` (for `SupabasePolicyRepository` in a real deployment). No policy-engine-specific configuration exists, a policy's own content is the only per-policy configuration surface.

Chapter 5: Signal-Intent Binding and Signal-State Verification

What it is

Two separate, deliberately independent checks that run before `PolicyEngine.evaluate` ever sees a request's signals:

1. **Signal-Intent Binding** (`SignalIntentBinder`, `packages/policy/src/SignalIntentBinder.ts`), proves a policy's declared signals describe the *same action* as the Intent that will actually execute if the transaction is approved.
2. **Signal-State Verification** (`SignalStateVerifier` port, `packages/policy/src/types/SignalStateVerifier.ts`, composed via `CompositeSignalStateVerifier`), proves those signals are *true*, not merely consistent.

Both exist because a caller-declared signal, on its own, only ever proves what the caller *says*. Neither check trusts the caller; each closes a different gap in what a bare signal can prove.

Why it was built

`PolicyEngine` evaluates whatever `PolicySignals` it's handed. Nothing about `PolicyEngine` itself prevents a caller from declaring `{"managerApproved": true}` when no manager ever approved anything, or from declaring a small, fully-verified signals payload while the real `Intent` (the thing that actually gets signed and executed) targets something entirely different. `SignalIntentBinder`'s own doc comment states the risk directly (`Policy.ts:238-249`): without it, "a caller could otherwise declare a small, fully-verified signals payload while Intent silently targets something else entirely, and receive a signed APPROVED trust record for it." `SignalStateVerifier`'s doc comment (`SignalStateVerifier.ts:24-33`) draws the companion distinction explicitly: "SignalIntentBinder proves a policy's signals describe the same action as Intent; it never proves those signals are *true*." Two different failure modes, two different, additive mechanisms.

How it works

Signal-Intent Binding

A policy's `boundSignals` field (`Policy.ts:266`, e.g. `{"paymentAmount": "parameters.amount"}`) declares that a given signal key must equal the value found at a specific dot-path into the real `Intent`, an `IntentSnapshot` of `{ target, parameters }` (`SignalIntentBinder.ts:9-12`).

`SignalIntentBinder.findViolations(policy, signals, intent)` (`SignalIntentBinder.ts:67-95`) walks every `boundSignals` entry, resolves the dot-path against the real `Intent` via `resolveIntentPath` (`SignalIntentBinder.ts:32-44`, returns `undefined` for any missing path segment rather than throwing, so a missing Intent field is reported as a mismatch, not a crash), and compares it by strict equality (`!==`) against the caller-declared signal value. Any mismatch becomes a `SignalIntentBindingViolation`, the empty array means either the policy declares no `boundSignals` at all, or every declared binding held.

Only signals with a genuine Intent-side equivalent belong in `boundSignals`, an amount, a target/vendor identifier. A signal representing an external fact with no Intent-side equivalent (`vendorVerified`, `riskScore`) cannot be expressed this way and is deliberately left out, acknowledged instead in `unboundSignalReasons` (Chapter 4 covers the validator's enforcement of this).

Signal-State Verification

The `SignalStateVerifier` interface (`SignalStateVerifier.ts:42-47`) is a single async method: `findViolations(request, signals) => Promise<readonly SignalStateViolation[]>`. A `request` carries `action`, `businessTransactionId`, and `intentParameters`, enough for an implementation to decide whether, and how, to independently re-derive the relevant facts from a real external source and compare them against what the caller declared.

The interface is explicitly optional and capability-scoped: an implementation that doesn't recognize the given `action` returns an empty array, the same "nothing to check" shape `SignalIntentBinder` uses when a policy has no `boundSignals`.

`CompositeSignalStateVerifier` (`packages/policy/src/CompositeSignalStateVerifier.ts`) combines multiple capability-scoped verifiers into one: it queries each in the supplied order and returns the first non-empty result (`CompositeSignalStateVerifier.ts:23-35`). This lets `RuntimeEngine`, which accepts exactly one `SignalStateVerifier`, be wired with as many capability-specific verifiers as production needs, each responsible for only the action(s) it knows how to check.

A concrete production implementation: `createHubSpotSignalStateVerifier` (`packages/api/src/bootstrap/createHubSpotSignalStateVerifier.ts`) builds a `HubSpotSignalStateVerifier` (from `@parmana/connector-hubspot`) wired with the real signing key (`FileKeyProvider`, `DEFAULT_KEY_ID`), the real execution gateway, and an `ApprovalVerifier` for independently checking `preAuthorizedForAmountChange`, a signal that, without this, would be nothing but a caller's unverified claim. Since 2026-09-30 `hubspot-deal-update` 1.1.0 needs one signed approval for every update (`dealUpdateApproved`), checked by `ApprovalSignalVerifier`, and the HubSpot verifier skips its own pre authorization check whenever the policy declares `approvalSignals`, so one approval is not spent twice. Its own doc comment notes `approvalVerifier` is always supplied in production, never omitted; only direct unit tests construct the verifier without it.

Where both run

Both checks run inside the runtime pipeline (Chapter 8 covers the full order) before `PolicyEngine.evaluate`. A violation from either becomes an ordinary policy `REJECT`, no authorization is ever generated for a request whose declared signals don't match the real Intent, or don't match independently verified reality.

How it enables things, with a concrete example

- **Tutorial 62** (`examples/tutorials/62-signal-intent-binding/run.ts`) exercises `SignalIntentBinder` directly, including a deliberate mismatch between declared signals and real Intent parameters.
- **Tutorial 71** (`examples/tutorials/71-hubspot-signal-state-verification/run.ts`) exercises the real `HubSpotSignalStateVerifier` implementation described above.
- **Tutorial 82** (`examples/tutorials/82-composite-signal-state-verification/run.ts`) demonstrates `CompositeSignalStateVerifier` combining multiple capability-scoped verifiers.
- **Tutorial 98** (`examples/tutorials/98-signal-freshness-enforcement/run.ts`) covers a related freshness concern at the execution boundary, worth reading alongside this chapter, though it's a distinct mechanism.

How to validate this yourself

- `packages/policy/src/SignalIntentBinder.ts` (96 lines) and `packages/policy/tests/unit/SignalIntentBinder.test.ts`.
- `packages/policy/src/types/SignalStateVerifier.ts` and `packages/policy/src/CompositeSignalStateVerifier.ts` for the port and its composition.
- `packages/api/src/bootstrap/createHubSpotSignalStateVerifier.ts` for a real, wired production implementation, and its corresponding package under `packages/connector-hubspot/` (or wherever `@parmana/connector-hubspot` actually lives in this checkout) for the verifier's own logic.

Integration requirements

`SignalIntentBinder` needs nothing beyond the policy and the real Intent, both already present in any request. A capability-specific `SignalStateVerifier` (like the HubSpot one) needs whatever real credentials that connector needs to make its own independent verification calls (a HubSpot private app token, in that case), see the connectors chapter for the full credential list per connector.

Chapter 6: Capability and Policy Binding

What it is

`CapabilityPolicyBinder` (`packages/capability-registry/src/CapabilityPolicyBinding.ts`) is a one-method class that enforces a single invariant: for a fixed set of production capabilities (a connector action like `hubspot:deal-update` or `paytm:refund`), the policy the caller declares must be the one canonical policy Parmana designates for that capability, never a different, caller-chosen one.

It lives in its own package, `@parmana/capability-registry`, deliberately depending on nothing except `@parmana/shared`, not on `@parmana/policy`, not on either connector package, so that anything needing to ask "is this capability bound, and to what" can depend on just this map, and `@parmana/policy` itself can depend on it without creating a dependency cycle back through a connector package.

Why it was built

This is a distinct problem from Chapter 5's signal-intent binding, easy to conflate with it because both involve the word "binding." Signal-intent binding proves a policy's declared signals match the real Intent *once a policy has already been selected*. Capability-policy binding is about *which policy gets selected in the first place*.

`CapabilityPolicyBinding.ts`'s own doc comment (lines 74-96) states the gap directly: a capability's `boundSignals` / `SignalStateVerifier` protections are scoped to one specific policy. Nothing upstream enforces that the request is actually evaluated against *that* policy, `PolicyRouter` / `FilePolicyRepository` load whatever `policy.name` / `policy.version` the caller declares, `PolicyEngine.evaluate` takes no `action` parameter at all, and the connector-level capability check only confirms the resolved connector declares the capability, never which policy authorized it. A caller could therefore pair a real capability (`hubspot:deal-update`) with an unrelated, unprotected policy (say, `vendor-payment/2.0.0` at the time, which declares no `boundSignals` for that capability at all), have that unrelated policy's looser rules evaluated against self-declared signals, and get the real `hubspot:deal-update` intent executed anyway, bypassing the capability's intended protections entirely, not merely weakening them.

Found by an independent repository verification (Phase 2K, TD-22) and closed as its own package boundary was chosen).

How it works

The canonical binding table

`CANONICAL_CAPABILITY_POLICY_BINDINGS` (`CapabilityPolicyBinding.ts`) is a `ReadonlyMap` from capability string to `PolicyReference` (`{ name, version, schemaVersion }`). The **name** is fixed in code. The **version** in the table applies only where policy governance is not enforced (`NODE_ENV` `test` and `development`). Where it is, production included, the version in effect is the one most recently approved through policy governance, so a new version goes live when it is approved, with no deploy. Current bindings, with the version used without governance:

Capability	Bound policy
`hubspot:deal-fetch`	`hubspot-deal-read` @ 1.0.0
`hubspot:deal-update`	`hubspot-deal-update` @ 1.1.0
`github:pr-fetch`	`github-pr-read` @ 1.1.0
`github:pr-merge`	`github-pr-approval` @ 1.1.0
`paytm:refund`	`customer-refund` @ 1.2.0
`slack:post-message`	`slack-post-message` @ 1.1.0

Every policy in this table needs a signed human approval for the action and its resource,

Every entry corresponds to a capability actually registered in production bootstrap (`packages/api/src/bootstrap/createConnectorRegistry.ts`) and the one policy file purpose-built to authorize it. An action with no entry here, every test/tutorial/example fixture action, and any future capability not yet given a canonical policy, is entirely unaffected by `CapabilityPolicyBinder`. The table is additive, not a replacement for

`PolicyRouter` / `PolicyEngine`; it doesn't change how a selected policy is loaded or evaluated, only whether the caller was allowed to select a different one.

Since 2026-09-28 a read and a merge on GitHub have separate policies, so a merge can need a signed approval for its pull request without every read needing one. In production each new version applies once it is approved through policy governance.

The request-time check

`await CapabilityPolicyBinder.findViolation(action, declared)` looks up `action` in the canonical table. No entry means nothing to enforce, returns `undefined`. Otherwise the declared name must equal the bound name, and the declared version must equal the version in effect: the one most recently approved for that name (`findMostRecentForName` on the approval records), where policy governance is enforced, or the table's version elsewhere. An older version is refused even if it was approved in the past; approving it again makes it current, which is how a rollback works. No approved version, or a failed lookup, refuses the request with a `reason`. Any mismatch returns a `CapabilityPolicyBindingViolation` (`{ action, expected, declared, reason? }`), which the runtime pipeline (Chapter 8) turns into a rejection before the (wrong) policy file is even evaluated.

The startup-time companion check

A second, separate mechanism enforces the other direction: that every capability a connector actually registers has *either* a canonical binding *or* an explicit, reasoned exemption. `assertConnectorCapabilitiesBound` (`packages/api/src/bootstrap/assertConnectorCapabilitiesBound.ts`) runs once, at startup, from `createConnectorRegistry.ts`, after every connector registration is built and before the registry is handed back to the rest of bootstrap. For each registered capability, it checks `CANONICAL_CAPABILITY_POLICY_BINDINGS` first, then falls back to `INTENTIONALLY_UNBOUND_CAPABILITIES` (`packages/api/src/bootstrap/intentionallyUnboundCapabilities.ts`), a second allowlist for capabilities deliberately left unbound, each entry carrying a reason. A capability found in neither map fails the process at startup, before it can bind a port. The file's own comment notes this assertion currently never fires outside a test-fixture exemption, since every real production capability already has a binding, its purpose is making sure the *next* connector added can't silently ship the same gap `CapabilityPolicyBinder` was built to close.

Together, the two checks form a closed loop: the startup check guarantees every real capability is accounted for (bound or explicitly exempted), and the request-time check guarantees a caller can't route a bound capability through the wrong policy.

How it enables things, with a concrete example

- **Tutorial 83** (`examples/tutorials/83-capability-policy-binding/run.ts`) exercises `CapabilityPolicyBinder` directly, including the exact TD-22 exploit shape (a real capability paired with an unrelated, unprotected policy) and confirms it's now rejected.
- **Tutorial 31** (`examples/tutorials/31-authorization-binding/run.ts`) covers a related but distinct binding concern at the execution-authorization layer, worth reading for contrast, not a substitute for Tutorial 83.

How to validate this yourself

- `packages/capability-registry/src/CapabilityPolicyBinding.ts` (124 lines, read the whole file, every design decision is explained in its own doc comments).
- `packages/capability-registry/tests/unit/CapabilityPolicyBinder.test.ts`, its own header comment restates the exact exploit this class closes, and the test suite proves it.
- `packages/api/src/bootstrap/assertConnectorCapabilitiesBound.ts` and `packages/api/src/bootstrap/intentionallyUnboundCapabilities.ts` for the startup-time companion check.
- `packages/api/src/bootstrap/createConnectorRegistry.ts` to see exactly where and when the startup check runs relative to connector registration.

Integration requirements

None beyond registering a connector at all, `CapabilityPolicyBinder` needs no configuration. Adding a new capability that should be protected this way means adding an entry to `CANONICAL_CAPABILITY_POLICY_BINDINGS`; deliberately leaving one unprotected means adding a reasoned entry to `INTENTIONALLY_UNBOUND_CAPABILITIES` instead, the startup assertion will fail the process if a new capability is registered with neither.

Chapter 7: Policy Governance and the Maker-Checker Flow

What it is

Policy Governance is the mechanism that controls how the content of a `policy.json` file is allowed to change. A policy change cannot take effect the moment someone writes it. It must first be proposed by one authenticated human (the maker), then reviewed and cryptographically approved by a second, genuinely different authenticated human (the checker), before the live policy content updates and a signed, durable approval record is created. The system enforces the maker and checker being different people at the API layer itself, not as a process convention someone could forget to follow.

Why it was built

Before this feature existed, policy authoring sat entirely outside Parmana's own trust model. external to the system's scope. In practice this meant any caller with write access to the `policies/` directory could change what a policy allows or refuses, with no second party involved and no durable, signed record of who made the change or why. Given that policy content decides real outcomes (who gets paid, what gets deployed, what an AI agent is allowed to do), that gap was treated as unacceptable once the system's scope grew to include policy authoring at all. Policy Governance is the answer: a policy change now goes through the same kind of two-person, cryptographically-verified control a bank uses for a wire transfer.

How it works, in full

The lifecycle

A `PendingPolicyChange` (`packages/shared/src/domain/pending-policy-change.ts`) moves through exactly three states, and only ever forward, never back:

```
PENDING_APPROVAL  ->  APPROVED
PENDING_APPROVAL  ->  REJECTED
```

Its fields capture the full proposal: `pendingPolicyChangeId`, the `policyName` and `policyVersion` it targets, the full `proposedContent` (the entire `policy.json`, never a diff or patch), `proposedBy` and `proposedAt`, a required `reason`, and once resolved, `resolvedBy`, `resolvedAt`, and (for a rejection) `rejectionReason`. The type deliberately types `proposedContent` as plain JSON rather than `@parmana/policy`'s `Policy` type, because `@parmana/shared` sits below `@parmana/policy` in the dependency graph and cannot import it without creating an import cycle. Structural validation against the real policy schema happens one layer up, in the API route, before a `PendingPolicyChange` is ever created.

The four endpoints

All four live in `packages/api/src/routes/pending-policy-changes.ts`, mounted at `/policies` in `packages/api/src/app.ts`.

Method	Path	Purpose	Who can call it
`POST`	`/:name/:version/pending-changes`	Propose a change (maker)	Any authenticated human caller (`credentialHolderType: USER`)
`GET`	`/pending-changes?status=...`	List changes, with a `diff` (`current` vs. `proposed`) for each	Any authenticated human caller
`POST`	`/pending-changes/:id/approve`	Approve (checker)	A human caller who is not the proposer, with a valid step-up envelope
`POST`	`/pending-changes/:id/reject`	Reject (checker)	Same as approve, plus a required `rejectionReason`

Every one of the four calls `requireHumanCaller()` first (`packages/api/src/routes/pending-policy-changes.ts`). That function checks `isHumanCaller(req.callerCredentialHolderType)`

(`packages/api/src/auth/isHumanCaller.ts`), which returns true only when the caller's credential was provisioned with `credentialHolderType: AuthorityType.USER`. An undefined value, or `ROLE` / `SERVICE` / `ORGANIZATION`, is denied the same as an unset one; the default is fail closed, never "assume human." A denial is recorded as a signed `caller.non_human_denied` audit event before the request is rejected with `NonHumanCallerDeniedError`. Notably, `isHumanCaller` is called only from these four handlers. It is not wired into the caller-auth middleware globally, so the core execution pipeline (an AI agent submitting a transaction, for example) stays fully actor-agnostic; only policy authorship itself is restricted to verified humans.

Maker cannot equal checker

Both the approve and reject handlers load the existing `PendingPolicyChange` by id and compare `existing.proposedBy === req.callerId` (`packages/api/src/routes/pending-policy-changes.ts` and `:605`). A match throws `SameActorCannotApproveOwnChangeError` immediately, before any step-up verification is even attempted. There is no override, no admin bypass, and no configuration flag for this check. It is the literal enforcement of "maker never equals checker."

Step-up authorization (Layer 4)

Passing `requireHumanCaller` and the maker/checker distinctness check is not enough to approve or reject. The request must also carry a `PolicyChangeStepUpAuthorization` envelope, checked by `requireStepUpAuthorization()` (`packages/api/src/routes/pending-policy-changes.ts`), which in turn delegates to `PolicyChangeStepUpVerifier` (`packages/api/src/auth/PolicyChangeStepUpVerifier.ts`). This is a second, independent cryptographic proof of intent, on top of (never instead of) the checker's ordinary bearer token.

The envelope is produced entirely offline, on the checker's own machine, by `PolicyChangeStepUpAuthorizationSigner` (`packages/crypto/src/PolicyChangeStepUpAuthorizationCrypto.ts:40`). Signing takes the `pendingPolicyChangeId` and the intended `action` ("approve" or "reject"), stamps a fresh `nonce`, `authorizedAt`, and an `expiresAt` (120 seconds later by default), and signs the whole payload with the checker's Ed25519 step-up private key: a key that is always a separate keypair from the checker's own bearer-token identity, generated independently (`ArtifactSigner` / `Ed25519SignatureProvider`, hardcoded to Ed25519 regardless of whatever signature algorithm the server's own runtime signing key uses, since a step-up key is operator-held and never depends on server-side config at all).

Verification, server-side, runs through `PolicyChangeStepUpVerifier.verify()` (`packages/api/src/auth/PolicyChangeStepUpVerifier.ts:154`), which composes two phases:

1. **`verifyChecks()`**: every side-effect-free check: payload version supported, signature verifies against `req.callerStepUpPublicKey` (the public key on file for the authenticated caller, from `PARMANA_API_KEYS`, never the envelope's own claimed keyId alone), not expired, `pendingPolicyChangeId` and `action` match what this specific request is for, and the envelope's own TTL (`expiresAt - authorizedAt`) does not exceed the server's configured maximum (120 seconds by default, `PolicyChangeStepUpVerifierOptions.maxTtlSeconds`).
2. **`consumeNonce()`**: only called if every check above passed, this burns the envelope's nonce in a dedicated `NonceStore` (`createPolicyChangeStepUpNonceStore.ts`, `SupabasePolicyChangeStepUpNonceStore` in production), so the exact same signed envelope can never be replayed for a second approval. This nonce namespace is entirely separate from execution-authorization or approval-artifact nonces.

A failing check throws `StepUpAuthorizationInvalidError`. The specific reason (missing envelope, expired, replayed, wrong id, wrong action, bad signature, no step-up key provisioned at all) is logged server-side via `console.error` only, and never reaches the HTTP response body: the caller only ever sees a generic 403 with code `STEP_UP_AUTHORIZATION_INVALID`. This is deliberate: the approve/reject endpoint is reachable by any authenticated human other than the proposer, not just the pending change's intended checker, so a granular failure breakdown would hand an attacker holding a stolen bearer token useful reconnaissance (for example, whether a given caller even has a step-up key provisioned) they have no legitimate need for. A real checker debugging their own signing setup has the detail available locally, from the tool that produced the envelope in the first place.

What happens on approve

Once every check passes, the approve handler calls `PolicyChangeApprovalService.approve(existing, req.callerId)` (`packages/api/src/governance/PolicyChangeApprovalService.ts`) *before* marking the pending change `APPROVED`. This ordering is load-bearing: if the service throws partway through, the pending change remains untouched at `PENDING_APPROVAL`, never falsely marked resolved with no corresponding live effect. Inside `approve()`:

1. Loads the current live content at the write target (`proposedContent.policyVersion`, not the pending change's own `policyVersion` field: those can legitimately differ; see `PendingPolicyChange`'s own doc comment on why an in-place patch to an existing version and a version bump are both allowed proposals).
2. Computes `contentHashAfter` (a hash of the proposed content) and, if a prior version existed, `contentHashBefore`.
3. Looks up the most recent `PolicyChangeApprovalRecord` for this same `(policyName, writeVersion)`, if any, and computes `previousRecordHash`: this chains each new record to the one before it, so a deleted or reordered record in the approval history becomes detectable (`verifyPolicyGovernanceIntegrityAtStartup`'s "chain-broken" check, below).
4. Signs the resulting `PolicyChangeApprovalRecord` and persists it via `policyChangeApprovalRecordRepository.create()`: **before** writing the live file.
5. Only then calls `policyRepository.save(policyName, writeVersion, proposedContent)`.

The record-before-file ordering (step 4 before step 5) means the recoverable failure mode is a persisted record with no corresponding file write yet, which a later integrity check can compare against and flag as `"missing"`. The alternative order would risk a live policy silently changing with no durable evidence anything approved it, a governance gap with nothing to point an integrity check at.

Rejections never touch `PolicyChangeApprovalService` at all. A rejection is purely a repository state transition (`resolve()` with `outcome: "rejected"` and the required `rejectionReason`); the rejection reason itself is the durable evidence a rejection leaves behind, since there is no live effect to sign evidence about.

Where the approved content actually lives: two `PolicyRepository` implementations

`PolicyChangeApprovalService.save()` writes through the `PolicyRepository` interface (`packages/policy/src/PolicyRepository.ts`), which has two real implementations:

- **`FilePolicyRepository`** (`packages/policy/src/FilePolicyRepository.ts`): writes `policy.json` to local disk at `<basePath>/<name>/<version>/policy.json`, via a temp-file-then-atomic-rename pattern. Used for local development and tests, where the filesystem is always writable.
- **`SupabasePolicyRepository`** (`packages/policy/src/SupabasePolicyRepository.ts`): It stores `(policy_name, policy_version) -> content_json` rows in a `policies` table (the policies table migration) via a direct Postgres connection.

Serverless functions run on a **read-only filesystem**, so policy content cannot be written to local files in a deployment. `application.ts` chooses the policy repository from `config.storage.provider`: memory or test configurations get `FilePolicyRepository`, and any real database configuration gets `SupabasePolicyRepository`.

Deploy-time and periodic integrity checking

At startup the check is a gate. `assertPolicyGovernanceIntegrityAtStartup()` runs before the port binds, and the server does not start on a mismatch or if the check cannot run. The periodic re-run described below stays fail-open and logs.

Two further mechanisms exist to catch a governance bypass after the fact: a direct edit to `policies/{name}/{version}/policy.json` that skips the pending-change API entirely:

- **`verifyPolicyGovernanceIntegrityAtStartup()`** (`packages/api/src/governance/verifyPolicyGovernanceIntegrityAtStartup.ts`) compares every approved `(policyName, policyVersion)`'s live content against its most recent approval record's `contentHashAfter`, and separately checks that each record's `previousRecordHash` correctly chains to the record before it. It is deliberately **fail-open**: it never throws, never blocks startup, and every outcome (clean pass, mismatch, or "could not run at all") is logged loudly and distinctly, so "nothing to report" and "couldn't check" never look the same in deploy logs. `schedulePolicyGovernanceIntegrityCheck()` re-runs the same check every 5 minutes for the life of the process, using an `.unref()`'d timer so it can never itself keep the process alive past shutdown.
- **`scripts/verify-policy-changes-approved.ts`** is the opposite discipline: **fail closed**, meant to gate a CI run or a manual pre-deploy check, not to run inside the live server. Given a list of changed policy files (or `--full-scan`), it hashes each file's live content and checks it against `policy_change_approval_records` via a read-only Supabase `anon` credential, scoped by RLS to that one table. A missing approval record, a content-hash mismatch, and any failure to complete the check at all (Supabase unreachable, a malformed response) are all treated identically as a failure: there is no "couldn't check, let it through" path. `.github/workflows/ci.yml`'s `verify-policy-approvals` job runs it with `--full-scan` on every push and pull request.

How it enables things, with concrete examples

- **`examples/tutorials/103-policy-governance-maker-checker/run.ts`** : the full manual flow against a real, locally-running Express server: a maker proposes, a `SERVICE`-credentialed caller is denied from proposing at all, the maker is denied from approving its own proposal, a distinct checker with no step-up envelope is denied, and finally a distinct checker with a valid envelope succeeds: proving the live `policy.json` file gets written and a signed approval record gets persisted only once every layer has passed.
- **`examples/tutorials/116-supabase-policy-repository/run.ts`** : proves `SupabasePolicyRepository` satisfies the exact same `PolicyRepository` contract as `FilePolicyRepository`, hermetically, against a minimal fake `pg.Pool` (no real network or database). Also reproduces the real `EROFS` failure directly, against a genuinely read-only temp directory, on platforms where the OS actually enforces it.
- **`examples/tutorials/117-maker-checker-one-shot-scripts/run.ts`** : exercises `scripts/local-review-action.ts` (sign and submit an approve/reject for an existing pending change in one process) and `scripts/refresh-approved-policy-content.ts` (propose, sign, and submit in one process, always from the current on-disk file), and proves both produce the identical durable effects as the fully manual flow. These scripts exist because chaining a hand-signed, 120-second-lived envelope across separate notes below.

How to validate this yourself

- The four route handlers and their tests: `packages/api/src/routes/pending-policy-changes.ts`, and `packages/api/tests/integration/pending-policy-changes-governance.integration.test.ts` (23 cases: human-only enforcement on all four endpoints, maker-checker distinctness on approve/reject, every step-up failure mode, file-write-plus-signed-record content, and path-traversal rejection on a malicious `proposedContent.policyVersion`).
- `packages/api/src/governance/PolicyChangeApprovalService.ts` and its unit test `packages/api/tests/unit/PolicyChangeApprovalService.test.ts`, which injects a failure at each step independently and confirms the ordering guarantee described above.
- `packages/api/src/auth/isHumanCaller.ts` and `packages/api/tests/unit/isHumanCaller.test.ts`.
- `packages/api/src/auth/PolicyChangeStepUpVerifier.ts` and `packages/crypto/src/PolicyChangeStepUpAuthorizationCrypto.ts`, plus `packages/api/tests/unit/verifyPolicyGovernanceIntegrityAtStartup.test.ts` (7 cases) for the integrity checker specifically.
- To actually run the real flow against a real deployment, use Policy lifecycle and approvals (the full lifecycle, every command, and a troubleshooting table) and Credentials for maker, checker, approver and agent. This chapter explains the system; those two are the how to.

Integration requirements

- **Provisioning a checker credential**, one time, per checker:

  ```
  npx tsx scripts/generate-api-key.ts --caller-id "<checker-name>" --credential-holder-type USER --g
  ```

  This prints a bearer key, a step-up **private** key (PEM, never written to disk by the script itself), and an `entry` JSON block containing only the key **hash** and the step-up **public** key: safe to store or transmit, unlike the other two. The operator appends `entry` to the `PARMANA_API_KEYS` environment variable (a JSON array); the checker keeps the bearer key and the private key file on their own machine.
- **The `PARMANA_API_KEYS` entry shape** for a checker, concretely:

  ```
  {
    "callerId": "checker-name",
    "keyHash": "<64-character lowercase hex sha256 of the bearer key>",
    "credentialHolderType": "USER",
    "stepUpPublicKey": "-----BEGIN PUBLIC KEY-----\n...\n-----END PUBLIC KEY-----\n"
  }
  ```

- **`PARMANA_STORAGE`** determines which `PolicyRepository` implementation is live: anything other than `memory` (and `NODE_ENV !== "test"`) selects `SupabasePolicyRepository`, which additionally requires `DATABASE_URL`.
- **`packages/governance-ui`** is an optional, small, server-rendered Express app covering propose, list, and diff-review only. It deliberately has **no** approve or reject routes: its diff page instructs a checker to run `scripts/sign-policy-change-step-up.ts` (or one of the one-shot scripts) locally and submit the result themselves, with their own bearer token, entirely outside the UI. Collecting a step-up private key in a web UI

would defeat the one property this whole mechanism exists to guarantee: that the key never leaves the checker's own machine.

What remains genuinely open

Execution time policy governance verification is enforced. `RuntimeEngine` refuses execution against any policy with no approval record, an invalid approval-record signature, or content that no longer matches its approval record, and `ExecutionGateway` runs the same check again at release, requiring the approved hash, the hash signed into the authorization, and the live hash to agree §2.35 and §2.36). Since 2026-09-20 this is on everywhere except when `NODE_ENV` is exactly `test` or `development`. `POLICY_EXECUTION_VERIFICATION_ENFORCED` is now only an opt in for those two environments and cannot switch enforcement off in production. The consequence for operators is that every policy a deployment executes against must have a genuine signed approval record before that deployment is promoted, or executions under it are refused.

Chapter 8: The Runtime Pipeline

What it is

The Runtime Pipeline is the ordered sequence of checks and steps a `BusinessTransaction` passes through, from arrival to a signed `ExecutionTrustRecord`. Its center is `RuntimeEngine.execute()` (`packages/runtime/src/RuntimeEngine.ts`), a single method that loads a policy, runs every configured pre-authorization protection in a fixed order, evaluates the policy, builds a `Decision`, signs an `ExecutionAuthorizationPayload` if approved, and hands the result to two further pipelines (`RuntimePipeline` for execution, `BusinessTrustPipeline` for the final trust record). Everything downstream of a request being accepted runs through this one method.

Why it was built

A system that authorizes real-world actions needs one place where every protection is guaranteed to run, in a guaranteed order, with no way for a caller or a misconfigured deployment to skip a step silently. `RuntimeEngine` is that place. Its own class doc comment states its responsibilities directly: load the requested policy, evaluate it deterministically, create the `Decision` artifact, create the initial `Execution` artifact, execute the Runtime Pipeline, and produce the `ExecutionTrustRecord`. Most of the protections wired into it (signal-intent binding, capability/policy binding, signal-state verification, policy governance verification) were added incrementally, each as an **optional, trailing constructor parameter**, specifically so every pre-existing call site keeps compiling and behaving identically when a new protection is introduced. This is a deliberate, repeated pattern in this codebase, not an accident of growth.

How it works

`RuntimeEngine`'s constructor takes ten required dependencies (`RuntimePipeline`, `PolicyRouter`, `PolicyEngine`, `SignalIntentBinder`, `DecisionBuilder`, `ExecutionGate`, `ExecutionBuilder`, `BusinessTrustPipeline`, `RuntimeAuthorizationSigner`, `authorizationTtlSeconds`) and eight optional trailing ones (`hooks`, `refusalRecordBuilder`, `refusalRecordRepository`, `signalStateVerifier`, `capabilityPolicyBinder`, `policyExecutionVerifier`, `policyGovernanceAnchorResolver`, `signingReadiness`). At construction, it logs which optional protections are actually configured (`runtime_engine_constructed`), which is how an operator can confirm, from log output alone, exactly which protections are active for a given deployment.

`execute(transaction)` runs these steps, in this exact order (comments quoted verbatim from `RuntimeEngine.ts`):

1. **Policy load.** `policyRouter.load(transaction.policy.name, transaction.policy.version)`. `beforePolicyLoad` / `afterPolicyLoad` hooks fire around this.
2. **Policy content hash.** `policyContentHash = policyContentHasher.hash(policy)`, computed from the *actually loaded* policy document, never from what the caller declared, using the same `TrustRecordHasher` (canonicalize, then SHA-256) every other content hash in this codebase uses.
3. **Policy Governance evidence anchor.** If `policyGovernanceAnchorResolver` is configured, it resolves whether this policy has a valid, matching approval record. Purely evidentiary, a resolver error is caught and logged, never allowed to affect the real outcome.
4. **Policy Governance execution-time verification.** If `policyExecutionVerifier` is configured, it checks the same approval-record question, but this one *can* reject: "a policy with no approval record, an approval record whose signature does not verify, or live content that no longer matches its approval record is rejected before `PolicyEngine` ever evaluates a single rule in it."
5. **Capability/Policy binding (TD-22).** If `capabilityPolicyBinder` is configured and step 4 found no violation, it checks whether the invoked capability has a canonical policy binding and, if so, whether the declared policy matches it. Runs before signal-intent binding "for the same reason capabilityPolicyBinder runs before signalIntentBinder: checking a narrower guarantee against an already-wrong policy is meaningless."
6. **Signal-intent binding.** If steps 4 and 5 found no violation, `signalIntentBinder.findViolations(policy, signals, {target, parameters})` checks that every `boundSignals` entry actually matches the real Intent's target/parameters.
7. **Policy evaluation.** If none of steps 4 through 6 rejected, `policyEngine.evaluate(policy, signals)` runs the real rules. Any rejection from steps 4 through 6 becomes a synthetic `PolicyDecision` with `outcome: REJECT`, `evaluatedRules: 0`, no rule is ever evaluated for a request that fails an earlier check.
8. **Signal-state verification.** Only runs if the provisional decision is `APPROVE`, "a request already rejected... needs no independent re-fetch of real state." If `signalStateVerifier` is configured, it independently re-

derives the declared facts from a real external source; a mismatch overrides the decision to `REJECT`.

9. **Decision.** `decisionBuilder.build(transaction, policyDecision)`.
10. **Refusal Record.** If the decision is not `APPROVED`, a `RefusalRecord` is built and persisted, but this write is explicitly never allowed to affect or block anything downstream; a failure here is only logged (`refusal_record_write_failed`), the method quote: "The refusal itself must never depend on its own evidence being writable."
11. **Enforce.** `executionGate.enforce(decision)`, this is the actual gate; a `REJECT` throws here and nothing past this point runs for a rejected transaction.
12. **Authorization.** Only reached on `APPROVE`. `signalsHash` is computed, then `authorizationSigner.sign(...)` produces the `SignedExecutionAuthorization` (Chapter 9 covers this envelope's exact fields).
13. **Execution + Runtime Context.** `executionBuilder.build(...)` and the `RuntimeContext` are assembled, the context carries a *copy* of `transaction.policy` augmented with `contentHash` and, if resolved, `governanceAnchor`; the original caller-submitted transaction (already persisted before this method ever ran) is never mutated.
14. **Signing readiness, then the Execution Intent, then the Runtime Pipeline, then the Business Trust Pipeline.** If `signingReadiness` is configured, `assertReady()` runs first and throws `SigningUnavailableError` (`503 SIGNING_UNAVAILABLE`) when the evidence signing path cannot produce a signature that verifies, so nothing is released. Then, if `executionIntents` is configured, `executionIntents.prepare(context)` signs the Execution Intent and stores it, and throws `ExecutionIntentUnavailableError` (`503 EXECUTION_INTENT_UNAVAILABLE`) when it cannot, so nothing is released. Then `pipeline.execute(context)` runs the actual execution stages (Chapter 10 covers `ExecutionGateway`, one implementation of the `ExecutionSystem` interface this pipeline calls into). If that raises an error, the intent is marked `ERRORED`. When it returns, `markReleased(...)` saves the execution context on the intent. Then `trustPipeline.execute(...)` produces the final, signed `ExecutionTrustRecord`, and after `Runtime.execute()` stores it the intent is marked `FINALIZED`.

Signing readiness and failure after release

`signingReadiness` is `CachedSigningReadiness` in production. Its probe is `VerificationCrypto.probeSigning()`, which signs a synthetic artifact padded past 4096 bytes through the same `Signer` and key id used for trust records, then verifies it against the public key that `Signer` publishes. A success is trusted for 60 seconds, a failure is never cached, and concurrent requests share one probe. It is enforced by `createSigningReadiness()` everywhere except when `NODE_ENV` is exactly `test` or `development`.

Everything after `pipeline.execute()` returns happens after the action was released to the connector. A failure there, in the trust record pipeline, the `afterExecution` and `beforeTrustRecord` hooks, or persisting the record in `Runtime.execute()`, is thrown as `ExecutionRecordIncompleteError` (`500 EXECUTION_RECORD_INCOMPLETE`), which names the `businessTransactionId` and `authorizationId`, and a critical `execution_released_record_failed` or `execution_released_record_persist_failed` event is logged. A failure before release, such as a policy rejection, keeps its own error.

Execution Intents: signed evidence before release, and a repair path

The Execution Trust Record contains the execution result, so it can only be built after release. adds a separate record that does not: the Execution Intent. `ExecutionIntentService.prepare()` (`packages/runtime/src/ExecutionIntentService.ts`) builds it with `ExecutionIntentBuilder`, signs it with `ExecutionIntentCrypto` (the same signing key as the Trust Record, so KMS in production), and stores it, immediately before `pipeline.execute()`. It is fail closed: if the intent cannot be signed and stored, nothing is released and the caller gets `503 EXECUTION_INTENT_UNAVAILABLE`.

The signed fields are the ids, the policy reference, `policyContentHash`, `signalsHash` and `businessTransactionHash` copied from the signed authorization, `action`, `target` and `createdAt`. The result and the raw intent parameters are never in it. An intent proves what was about to be released. It does not prove the action was released or what its result was.

The intent has an unsigned operational state: `PREPARED` (signed and stored), `RELEASED` (the release stage returned and the execution context was saved), `FINALIZED` (a signed Trust Record exists) and `ERRORED` (the release stage raised an error, so the outcome is unknown). `markReleased`, `markErrored` and `markFinalized` are best effort and never throw, because they run after release. They log at critical severity when they fail. Marking an intent `FINALIZED` deletes the saved execution context, since the Trust Record then holds it. A fifth state, `RESOLVED`, is set only by `ExecutionIntentService.resolve()`: a verified human closes a `PREPARED` or `ERRORED` intent after reconciling it at the connector, recording `resolution` (`NOT_EXECUTED` or `EXECUTED`), a required note, who and when. It is idempotent, never calls a connector, refuses `RELEASED` and `FINALIZED` intents and any transaction that has a Trust Record, and is an unsigned operator statement, not a Trust Record.

`ExecutionIntentFinalizer` (`packages/runtime/src/ExecutionIntentFinalizer.ts`) rebuilds a missing Trust Record from the saved context. It never calls a connector, it is idempotent, and it refuses with `409 EXECUTION_INTENT_RESULT_NOT_RECORDED` when no context was saved. It is exposed as `POST /execution-intents/{businessTransactionId}/finalize` for a verified human credential, and it also verifies the record and generates the receipt. Enforcement is the rule `createExecutionIntents()` applies: on everywhere except `NODE_ENV` `test` or `development`, where `EXECUTION_INTENTS_CHECK=true` turns it on. The full operator procedure is in the docs site page Execution Intents.

`RuntimeFactory.create()` (`packages/runtime/src/RuntimeFactory.ts`) is the composition root that assembles a fully wired `RuntimeEngine` (via `RuntimeBuilder`) plus the surrounding `ExecutionTrustApplication` (transaction/execution/verification/receipt services). It takes the same optional protections as trailing parameters and only wires them into `RuntimeBuilder` when supplied (`if (signalStateVerifier) { builder.withSignalStateVerifier(...) }`), the same "absent means unconfigured, not broken" discipline as the constructor itself.

How it enables things, with examples

- `examples/tutorials/03-runtime-execution`, the baseline: a transaction through `RuntimeEngine.execute()` to a trust record, no optional protections.
- `examples/tutorials/16-runtime-pipeline`, the pipeline stages themselves.
- `examples/tutorials/18-runtime-hooks`, the `RuntimeHook` interface (`beforePolicyLoad`, `afterDecision`, etc.) that lets an integrator observe or extend the pipeline without modifying `RuntimeEngine` itself.
- `examples/tutorials/19-runtime-composition`, composing multiple pipeline stages.
- `examples/tutorials/15-custom-runtime-component`, writing a custom pipeline stage.

How to validate this yourself

- `packages/runtime/src/RuntimeEngine.ts`, the method itself; every ordering claim above is a direct comment in this file.
- `packages/runtime/src/RuntimeFactory.ts`, `RuntimeBuilder.ts`, how a `RuntimeEngine` actually gets constructed for a real deployment. content-hash-at-decision-time assertion against real on-disk policy content.
- `packages/runtime/tests/unit/optional-protections-logging.test.ts`, proves the construction-time log line accurately reflects what's wired.
- `packages/runtime/tests/integration/runtime.integration.test.ts`, the fuller integration surface.

Integration requirements

None beyond what Chapter 2 (Configuration and Bootstrapping) already covers, `RuntimeEngine` itself takes no environment variables directly; every dependency it needs is constructed and passed in by `RuntimeFactory` / `application.ts`. The optional protections (`signalStateVerifier`, `capabilityPolicyBinder`, `policyExecutionVerifier`, `policyGovernanceAnchorResolver`) each have their own configuration surface, covered in the chapters specific to them.

Chapter 9: The Execution Authorization Envelope

What it is

A `SignedExecutionAuthorization` is the cryptographic proof that Parmana decided a specific request should execute. It's what crosses the boundary between "Parmana decided APPROVE" and "a connector actually did something in the real world." Its own doc comment states the guarantee directly: "Enterprise systems should execute only requests carrying a valid, verified `SignedExecutionAuthorization`." Nothing downstream is trusted to re-derive that a decision was made; the envelope carries the proof itself.

Why it was built

Without a signed, independently verifiable authorization, a connector (or anything holding the ability to call one) would have to trust that whatever called it had genuinely gone through `RuntimeEngine`, with no way to check. The envelope makes "was this actually authorized, by this exact decision, under this exact policy content, for this exact content, within this time window" a question anyone downstream can answer themselves, from the envelope alone, without a network call back to Parmana. Every field on the payload (`packages/shared/src/domain/execution-authorization.ts`) exists to close one specific gap

How it works

`ExecutionAuthorizationPayload` (the signed content) carries:

Field	Purpose
`version`	Must be `1`. A verifier rejects any other value, including a missing field, before even attempting signature verification.
`authorizationId`	Unique identifier for this authorization.
`nonce`	Single-use. A receiving system must reject a previously-seen nonce, replay protection.
`decisionId`	The approved `Decision` this authorization corresponds to.
`businessTransactionId`	The transaction.
`policyName` / `policyVersion`	Which policy produced the decision.
`authorizedAt` / `expiresAt`	ISO-8601 UTC. A receiving system must reject an expired authorization.
`businessTransactionHash`	Canonical hash of the `ExecutableContent` (`businessTransactionId`, `action`, `target`, `parameters`) approved for execution, computed identically on the signing side (from the runtime's in-memory transaction) and the verifying side (from the JSON-parsed request), so a mismatch means the payload was modified after signing while keeping the same `businessTransactionId`.
`policyContentHash` (optional)	The content hash of the *exact policy document* that produced this decision, not merely the version string. Optional so authorizations signed before this field existed keep verifying, absent means "not covered by the policy-freshness check," never "policy is stale."
`signalsHash` (optional)	Canonical hash of the runtime signals evaluated for this decision, the real-world conditions (vendor status, risk exposure, etc.) that justified it, distinct from both the content hash and the policy hash.
`submittedBy` (optional)	The authenticated caller identity, when caller-auth was enabled.
`grantedCapability` (optional)	The capability the caller-to-capability check confirmed this caller was permitted to invoke, carried forward as a signed fact rather than discarded once the request passes, a receiving system can confirm this equals the executed action, catching any divergence between what a caller was cleared for and what's actually presented for execution.

The complete envelope (`SignedExecutionAuthorization`) wraps this payload with `signature` (over the canonical serialization of the payload, see Chapter 3 for `CanonicalSerializer`), `keyId` (so a verifier can select the right public key), and `algorithm`.

Signing happens in `RuntimeAuthorizationSigner` (`packages/runtime/src/RuntimeAuthorizationSigner.ts`), which composes `@parmana/crypto`'s `AuthorizationSigner` against whichever `Signer` (`LocalFileSigner` or `KmsSigner`, resolved once through `SignerBootstrap`, see Chapter 3) is configured, using a per-tenant key resolved through `TenantKeyResolver` (falling back to the shared `"default"` key when no tenant-specific key exists). `RuntimeEngine.execute()` calls this once, at the "Authorization" step (Chapter 8, step 12), only ever reached after policy evaluation approved the request.

Verification happens twice, in two different places, checking different things:

1. **`@parmana/envelope-verifier`'s `EnvelopeVerifier`**, used by `ExecutionGateway` (Chapter 10), checks the envelope itself: version, signature, expiry, TTL policy, then the `businessTransactionHash` recompute-and-compare, then (if wired) the policy-freshness and signal-freshness checks, then nonce consumption last (the only side-effecting check, run only once everything else has passed).
2. Anyone holding the receiving system's copy of Parmana's public key can independently re-verify the same signature offline, without calling back to Parmana at all, this is what Chapter 18 (Independent Verification) covers.

How it enables things, with examples

- `examples/tutorials/11-execution-authorization`, the payload itself, signed and inspected.
- `examples/tutorials/26-execution-authorization-verification`, independent verification.
- `examples/tutorials/27-authorization-expiration`, the `expiresAt` check rejecting a stale envelope.
- `examples/tutorials/29-authorization-tampering`, mutating a signed payload and watching signature verification catch it.
- `examples/tutorials/31-authorization-binding`, the `businessTransactionHash` binding content to the authorization.
- `examples/tutorials/41-expired-authorization`, `43-stolen-authorization`, further negative cases against the same envelope.

How to validate this yourself

- `packages/shared/src/domain/execution-authorization.ts`, the type definitions; every claim above about a field's purpose is quoted or closely paraphrased from this file's own doc comments.
- `packages/runtime/src/RuntimeAuthorizationSigner.ts`, the signing side.
- `packages/crypto/src/AuthorizationSigner.ts`, `AuthorizationVerifier.ts`, the shared signing/verification logic.
- `packages/envelope-verifier/src/`, `EnvelopeVerifier`'s own check ordering.
- `packages/runtime/tests/unit/execution-authorization-wiring.test.ts`, proves the fields get populated correctly from a real `RuntimeEngine.execute()` call.

Integration requirements

- A configured signer (`KEY_PROVIDER=local` with `PARMANA_KEY_DIR`, or `KEY_PROVIDER=aws-kms` with the relevant AWS configuration, see Chapter 3).
- `EXECUTION_AUTHORIZATION_TTL_SECONDS` (defaults to 120 per this repo's own `.env`) controls how long a signed authorization remains valid before a receiving system must reject it.
- A receiving system that wants to verify authorizations itself needs Parmana's public key, see Chapter 18 for the discovery mechanism (`GET /keys/:keyId`, `/.well-known/jwks.json`).

Chapter 10: The Execution Gateway

A correction, first

An older architecture document in this repository assumed `ExecutionGateway` lives at `packages/api/src/execution-gateway/ExecutionGateway.ts`. That path does not exist. The real, current location is `packages/execution-gateway/src/ExecutionGateway.ts`, a separate, top-level package, not something nested inside `packages/api`. This chapter is written directly from that real file.

What it is

`ExecutionGateway` implements `@parmana/execution-system`'s `ExecutionSystem` interface, the same seam `RuntimeFactory.create()` (Chapter 8) accepts any implementation of. Its own class doc comment calls it "the sole release boundary" for whichever `Connector` it's constructed with: no execution reaches a connector without passing through this gateway's full verification sequence first.

Why it was built

A signed `ExecutionAuthorizationPayload` (Chapter 9) proves a decision was made, but something still has to actually check that proof before letting a connector run, and that something needs to independently re-verify, not just trust, every claim the envelope makes, including claims about the world having stayed the same since the envelope was signed (the policy hasn't changed, the underlying signals haven't drifted). `ExecutionGateway` composes `@parmana/envelope-verifier`'s `EnvelopeVerifier` (signature/expiry/TTL/nonce) and adds two further checks of its own: a content-hash recompute-and-compare, and, when wired, policy-freshness and signal-freshness checks against the *current* state of the world, not just the state at authorization time.

How it works

Construction (`ExecutionGatewayOptions`) requires a `publicKey` (or a `keyProvider` for keyId-aware, rotation-safe lookup) and a `nonceStore`, and exactly one of a `connector` or an `executionControl` (constructing with both, or neither, throws immediately). Optional `policyRepository` and `signalStateVerifier` enable the two additive checks below; omitting either skips that specific check rather than failing closed on it.

`verify(request, now)` runs the checks in this exact order, quoted directly from the class doc comment, "Session 3's ordering rule: side-effect-free checks first, nonce consumed last and only on success":

1. `version -> signature -> expiry -> TTL policy` (all inside `EnvelopeVerifier.verifyChecks()`).
2. `businessTransactionHash` recompute-and-compare, only attempted if step 1 passed.
3. `policyStillCurrent` recompute-and-compare, required and fail closed. The gateway recomputes the *current* content hash of the named policy and compares it to what the authorization was signed under. An authorization with no `policyContentHash` is rejected, and if the policy no longer exists at all (e.g. replaced in place by a governed change), this is treated identically to a content mismatch, not a separate error case. A gateway built without a `PolicyRepository` and a `policyApprovalVerifier` refuses to construct, unless the explicit `allowUnverifiedPolicy: true` option is set, which production bootstrap never sets.
4. `policyGovernanceVerified`, required and fail closed. The live hash that just matched the signed hash is handed to the policy approval verifier, which checks that the policy's most recent signed `PolicyChangeApprovalRecord` exists, that its signature verifies, and that its `contentHashAfter` equals that same hash. The approved hash, the signed hash and the live hash must all agree. A missing record, a bad signature, a differing hash, or an error from the verifier all reject the request.
5. `signalsStillCurrent` recompute-and-verify (when a `SignalStateVerifier` is wired and both the request and authorization carry signals/`signalsHash`), first a hash comparison (catching a request whose signals were altered after authorization), then an independent re-verification of those signals against real-world state via the verifier.
6. **Nonce consumption, last, only if every prior check passed.** This is the only side-effecting check in the sequence, "a mismatched or forged request must not burn a nonce."

`execute(request)` calls `verify()`, and on any failure throws, specifically, `NonceAlreadyConsumedError` when nonce replay is the *sole* failing check (every other check passed), or a generic `Error` naming every failing check and both hash values on a content mismatch otherwise. `isSoleFailureNonceReplay()` is the precise logic for that distinction: strip out `nonceUnseen`, and check every remaining value is `true` or `undefined` (an undefined check

means "skipped," which still counts as passing for this purpose, and only occurs under the legacy `allowUnverifiedPolicy` opt out, because in the default mode `valid` requires `policyStillCurrent` and `policyGovernanceVerified` to be positively `true`). On success, the verified `executableContent` is deep-frozen and handed to either the legacy `Connector` interface (`this.connector.execute(...)`) or the newer `executionControl` path (a `service.execute(...)` call with a freshly minted or static `gatewayAuthentication` value, or an `ExecutionChannel.release(...)` call for the deprecated channel-based path).

How it enables things, with examples

- `examples/tutorials/34-execution-gateway`, the gateway directly.
- `examples/tutorials/33-execution-boundary`, the boundary concept: what does and doesn't cross it.
- `examples/tutorials/32-execution-pipeline`, the gateway inside the larger pipeline.
- `examples/tutorials/81-connector-execution-gateway`, a real connector behind the gateway.
- `examples/tutorials/86-gateway-attestation`, attestation semantics at this boundary.

How to validate this yourself

- `packages/execution-gateway/src/ExecutionGateway.ts`, the full verification sequence; every ordering and failure-handling claim above is a direct comment or method in this file.
- `packages/execution-gateway/src/GatewayVerificationResult.ts`, the structured result shape (`checks`, `hashMismatch`, `policyContentMismatch`, `signalsHashMismatch`, `signalDivergence`).
- `packages/execution-gateway/tests/unit/execution-gateway.test.ts`, the primary unit coverage.
- `packages/execution-gateway/tests/unit/execution-gateway.dilithium3.test.ts`, the same gateway logic under a post-quantum signature algorithm (see Chapter 3).
- `packages/execution-gateway/tests/unit/credential-non-exposure.test.ts`, a separate but closely related guarantee this package also carries, covered in full in Chapter 11 (Credential Isolation).

Integration requirements

- A public key (or `KeyProvider`) matching whatever signed the authorizations this gateway will verify.
- A `NonceStore` implementation, `MemoryNonceStore` for tests/tutorials, `SupabaseNonceStore` (via the storage layer, Chapter 13) for a real deployment, so replay protection survives a process restart.
- To enable policy-freshness checking: a `PolicyRepository` (Chapter 4/13) pointed at the same policy content the signing side used.
- To enable signal-freshness checking: a `SignalStateVerifier` (Chapter 5) capable of re-deriving the relevant real-world facts.
- Exactly one of a `Connector` implementation or an `executionControl` configuration, Chapter 12 covers what a real `Connector` looks like.

Chapter 11: Credential Isolation

What it is

Credential isolation is the guarantee that a connector, the code that actually calls an external service like HubSpot, GitHub, or Paytm, never holds a long-lived, reusable credential. Instead it receives a single-use, time-bounded session credential, minted fresh for one specific execution, resolved to the real secret only at the last possible moment, and destroyed immediately after use, whether that use succeeded or failed.

Why it was built

A connector is the part of this system closest to the outside world, and therefore the part If a connector held the platform's real HubSpot API key or GitHub App private key directly, a single flaw in connector code could leak that credential, and the leak would be reusable by whoever obtained it, for as long as the credential remained valid. Credential isolation removes that exposure at the architecture level: even a fully compromised connector can only ever hand an attacker a credential that is already spent, or one that is about to expire in seconds and is bound to an authorization that has already been consumed.

How it works

The mechanism lives in `packages/execution-control/src/`, specifically two classes:

`SessionCredentialVault` (`SessionCredentialVault.ts:18-25`) exposes three operations: `issue()`, `consume()`, and `revoke()`. Its in-memory implementation, `InMemorySessionCredentialVault`, wraps an underlying `CredentialVault` (the thing that actually holds the real, long-lived secret) and never itself stores the secret value. The class's own doc comment states this precisely: "issue() only confirms the underlying credential exists, it never resolves or stores the secret" (`SessionCredentialVault.ts:44-46`).

- `issue(connectorId, authorizationId)` calls `credentials.getCredential(connectorId)` once, purely to confirm a credential exists for that connector, then discards the resolved value immediately. It returns a `SessionCredential` object (`SessionCredentialVault.ts:10-16`) containing only an id, timestamps, and the connector/authorization it's scoped to, never the secret itself.
- `consume(sessionCredentialId)` is the only method that ever calls back into the underlying vault for the real value, and it does so fresh, at the moment of use. Before it does, it checks the session record for three failure conditions, each throwing if true: already revoked, already used, or expired (`SessionCredentialVault.ts:97-111`). It then marks the session `used = true` and returns the real credential exactly once. A second call to
- `revoke(sessionCredentialId)` marks a session dead, unconditionally.

`SessionCredentialSecureConnector` (`SessionCredentialSecureConnector.ts`) is what actually wraps a real connector's `execute()` call with this lifecycle. Its own `execute()` method (`SessionCredentialSecureConnector.ts:74-131`) does exactly this, in order:

1. Asserts the connector's policy allows this specific request (`policy.assertAllowed(...)`).
2. Calls `sessionCredentials.issue(...)`, obtaining a session credential id.
3. Inside a `try` / `finally`: calls `sessionCredentials.consume(...)` to resolve the real credential just before use, passes it straight into the wrapped executor's `execute()` call, and in the `finally` block, unconditionally calls `sessionCredentials.revoke(...)`, this runs whether the executor succeeded, threw, or the policy check itself threw after issuance.
4. Records exactly one audit event, `execution.completed` or `execution.rejected`, naming the connector, the session credential's id (never its value), the authorization it executed under, and the capability invoked. The credential's id is auditable; the credential's value never appears anywhere in the audit trail.

Because `consume()` throws on a second call, and `revoke()` runs unconditionally on every exit path via `finally`, a session credential can be used at most once, ever, no matter how the execution ends.

One subtlety worth being precise about, from the class's own doc comment (`SessionCredentialSecureConnector.ts:20-32`): this connector is also constructed with a `gatewayAuthentication` value proving it was configured with material signed by the Gateway's key at registration time. That value is frozen once, at construction, and reused for every execution this connector instance will ever handle, it is not, and cannot be, a per-request proof by itself. The actual per-request replay protection comes from a different layer, `ConnectorPolicy`'s single-use `GatewaySession` check, enforced one level up, before a `GatewaySession` is even

created. The two guarantees (registration-time authenticity, and per-request single-use) are deliberately separate mechanisms, not the same check reused twice.

How it enables things, with a concrete example

`packages/api/tests/integration/credential-isolation.integration.test.ts` proves this guarantee at the HTTP boundary, not just at the library level. Its own doc comment (`credential-isolation.integration.test.ts:10-19`) explains the split precisely: `packages/execution-control/tests/unit/session-credential-secure-connector.test.ts` already proves "a session credential is issued and destroyed exactly once per execution" by calling `SessionCredentialSecureConnector.execute()` directly; this integration test proves the exact same guarantee holds through a real `POST /execute` HTTP request against the real Express app, through the real `ExecutionGateway` / `SessionCredentialExecutionControl` / `ExecutionControlService` / `SessionCredentialSecureConnector` chain. It builds its own "inspectable" execution system (`packages/api/tests/bootstrap/createInspectableExecutionSystem.ts`) specifically to get a direct reference to the audit sink and the session credential vault, which the production bootstrap chain does not expose by design. The test asserts that after a successful `/execute` call, exactly one `execution.completed` audit event carries a `credentialId`, confirming a session credential really was issued and consumed, not skipped.

`packages/api/tests/integration/execution-failure.integration.test.ts` is the companion case: it proves the same revoke-on-every-exit-path guarantee when the underlying connector executor throws, not just on the success path.

How to validate this yourself

- `packages/execution-control/src/SessionCredentialVault.ts` and `SessionCredentialSecureConnector.ts`, the actual mechanism.
- `packages/execution-control/src/types.ts`, `CredentialVault`, `ExecutionCredential`, `SecureConnector`, `ConnectorPolicy` interface shapes.
- `packages/execution-control/tests/unit/session-credential-vault.test.ts`, `session-credential-secure-connector.test.ts`, `session-credential-execution-control.test.ts`, the library-level proofs, one per class.
- `packages/api/tests/integration/credential-isolation.integration.test.ts` and `execution-failure.integration.test.ts`, the HTTP-boundary proofs.
- `packages/api/tests/bootstrap/createInspectableExecutionSystem.ts`, how a test gets a direct handle on the audit sink and vault that production code never exposes.

Integration requirements

None from an operator's perspective, this is an internal architectural guarantee, not a configurable feature. A new connector wired into this system automatically gets session credential isolation by virtue of being registered behind `SessionCredentialSecureConnector`; there is no opt-out path documented in the source, and none should be added without a strong, explicit reason.

Chapter 12: Connectors

What it is

A connector is the piece of code that actually performs a real-world action, a HubSpot deal update, a GitHub pull request merge, a Paytm refund, a Slack message, after every governance check upstream (policy evaluation, signal verification, capability binding, execution authorization) has already approved it. Connectors are deliberately dumb: they validate requests, execute one already-authorized operation using an already-resolved credential (Chapter 11), and return a deterministic response. They never evaluate policy, never authorize anything, and never resolve credentials themselves.

Why it was built

Every governed action this system can take needs an actual integration with the outside world, and that integration needs to be swappable, testable in isolation, and structurally incapable of bypassing the governance layers above it. The `Connector` contract (`packages/connector-sdk/src/ConnectorTypes.ts:104-111`) exists to enforce that boundary in code, not just in documentation: its own doc comment states plainly that "a Connector never receives a request directly from the Runtime or AI, only from `SdkConnectorExecutor`, itself only reachable through execution-control's `SecureConnector`, itself only reachable through the Execution Gateway" (`ConnectorTypes.ts:99-102`).

How it works

The `Connector` interface itself is small:

```
export interface Connector {
  readonly connectorId: string;
  readonly capabilities: ConnectorCapabilities;
  execute(
    request: ConnectorRequest,
    context: ConnectorExecutionContext,
  ): Promise<ConnectorResponse>;
}
```

`ConnectorExecutionContext` (`ConnectorTypes.ts:85-89`) carries a `credential` (a `CredentialHandle`, already resolved by the session credential vault from Chapter 11, not a raw secret the connector has to go fetch), a `timeoutMs`, and a `requestedAt` timestamp.

Each real connector in this codebase splits into two files: a `*-Capabilities.ts` file in a dedicated `@parmana/connector-{name}` package (pure metadata: capability identifier constants and request/response DTOs, no execution logic at all), and an executable adapter under `packages/execution-gateway/src/connector-execution/Gateway{Name}Adapter.ts` that actually performs the HTTP call. This split is deliberate and consistent across every connector; each capabilities file says so explicitly in its own header comment (e.g. `HubSpotCapabilities.ts:1-5`, `GitHubCapabilities.ts:1-5`).

The four real, live connectors

Connector	Capability string(s)	Governing policy	Status
HubSpot	`hubspot:deal-fetch`, `hubspot:deal-update`	`hubspot-deal-read@1.0.0` (fetch), `hubspot-deal-update@1.1.0` (update)	Live
GitHub	`github:pr-fetch`, `github:pr-merge`	`github-pr-read@1.1.0` (fetch), `github-pr-approval@1.1.0` (merge)	Live
Paytm	`paytm:refund`	`customer-refund@1.2.0`	Live
Slack	`slack:post-message`	`slack-post-message@1.1.0`	Live
Razorpay	(historical)	(historical)	Removed 2026-08-12

This table's policy column is read directly from `packages/capability-registry/src/CapabilityPolicyBinding.ts`, `CANONICAL_CAPABILITY_POLICY_BINDINGS`, the same table Chapter 6 covers in depth. A request declaring `hubspot:deal-update` must be evaluated against exactly `hubspot-deal-update@1.1.0` (or the version most recently approved through governance), or it is rejected before any policy file is even loaded. Every policy in this table needs a signed human approval for the action and its resource, reads included (Human approval).

HubSpot (`packages/connector-hubspot/src/`): two capabilities, fetch and update a deal. `HubSpotSignalStateVerifier` (a separate file in the same package) independently confirms a caller-declared signal like "this deal's stage really is X" against HubSpot's own API rather than trusting the caller's assertion, the concrete example Chapter 5 (signal-state verification) points to.

GitHub (`packages/connector-github/src/`): fetch a PR's state, merge a PR. Uses a GitHub App JWT (`GitHubAppJwt.ts`) and an installation-token credential provider (`GitHubAppCredentialProvider.ts`, in `execution-gateway`) rather than a static personal access token.

Paytm (`packages/connector-paytm/src/`): exactly one capability, `paytm:refund`, by design. Its own doc comment is explicit that this is deliberate: "there is no `paytm:*` wildcard and no additional Paytm capability... declared here; adding one is a deliberate, separate milestone" (`PaytmCapabilities.ts:6-11`). Structurally different from the other three connectors in one respect: it never talks to Paytm's own API directly. `baseUrl` is required, not optional, because every request goes to a separate, trusted, out-of-process service (`example-agent`, a different repository, the "Pfinite" integration referenced elsewhere in this codebase's operational history), which is the only thing that ever holds a real Paytm merchant key (`PaytmCapabilities.ts:60-66`). The wire-level action string this remote service expects, `"paytm-refund"` (`PAYTM_AGENT_WIRE_ACTION`), is a pure transport-boundary translation; Parmana's own internal capability identity stays `"paytm:refund"` everywhere else and is never renamed to satisfy the remote service's naming convention (`PaytmCapabilities.ts:30-40`).

Slack (`packages/connector-slack/src/`): exactly one capability, `slack:post-message`, same "no wildcard, deliberate scope" discipline as Paytm (`SlackCapabilities.ts:6-11`).

How to validate this yourself

- `packages/connector-sdk/src/ConnectorTypes.ts`, the `Connector` contract itself.
- `packages/connector-hubspot/src/`, `connector-github/src/`, `connector-paytm/src/`, `connector-slack/src/`, each connector's capability metadata and DTOs.
- `packages/execution-gateway/src/connector-execution/Gateway{HubSpot,GitHub,Paytm,Slack}Adapter.ts`, the actual executable adapters.
- `packages/capability-registry/src/CapabilityPolicyBinding.ts`, the canonical capability-to-policy table.
- `packages/api/src/bootstrap/create{HubSpot,GitHub,Paytm,Slack}Connector.ts` and `assertConnectorCapabilitiesBound.ts`, how connectors get registered at startup, and the startup assertion that every declared capability has a canonical policy binding.
- Integration tests: `packages/api/tests/integration/hubspot-deal-update.integration.test.ts`, `github-pr-merge.integration.test.ts`, `paytm-refund.integration.test.ts`, each connector's real, HTTP-boundary proof, including its mock server (`MockHubSpotServer.ts`, `MockGitHubServer.ts`, `MockPaytmConnectorServer.ts`, `MockSlackServer.ts`) for hermetic testing without real vendor credentials.

Integration requirements

Each connector needs its own credentials configured, only when actually used:

- HubSpot: `HUBSPOT_PRIVATE_APP_TOKEN` (defaults to HubSpot's real API; tests point `baseUrl` at a local mock instead).
- GitHub: `GITHUB_APP_ID`, `GITHUB_INSTALLATION_ID`, `GITHUB_APP_PRIVATE_KEY`.
- Paytm: a `baseUrl` pointing at a deployed `example-agent` instance, required, with no default, since there is no direct-to-Paytm path at all.
- Slack: (see `createSlackConnector.ts` for the exact env var; defaults to Slack's real Web API base URL otherwise).

A connector with no credentials configured is simply not registered at startup rather than registered in a broken state, each `create*Connector.ts` bootstrap file logs an `*_connector_unavailable` event naming the missing configuration, visible in this codebase's own startup logs.

Chapter 13: The Storage Layer

What It Is

The storage layer is the set of repositories that give every durable Parmana concept (a business transaction, an execution trust record, a refusal record, a pending policy change, beyond a single process's memory. It is accessed through one interface, `StorageProvider` (`packages/storage/src/StorageProvider.ts`), with two real implementations selected at startup: an in-memory one for tests and local development, and a Postgres-backed one for anything real.

Why It Was Built

Parmana's whole value proposition depends on durable, independently verifiable evidence. A signed execution trust record that only exists in process memory and disappears on restart proves nothing to anyone after the fact. The storage layer exists to make every artifact this system produces outlive the process that produced it, without forcing every package that needs to persist something to independently decide how.

How It Works

The interface

`StorageProvider` exposes five repositories:

```
export interface StorageProvider {
  readonly businessTransactions: BusinessTransactionRepository;
  readonly trustRecords: ExecutionTrustRecordRepository;
  readonly refusalRecords: RefusalRecordRepository;
  readonly pendingPolicyChanges: PendingPolicyChangeRepository;
  readonly policyChangeApprovalRecords: PolicyChangeApprovalRecordRepository;
}
```

(`packages/storage/src/StorageProvider.ts`)

Two implementations, chosen by `StorageFactory`

`StorageFactory.create({ provider })` (`packages/storage/src/StorageFactory.ts`) switches on the configured provider:

Provider value	Class returned	Notes
`"memory"`	`MemoryStorageProvider`	Five in-memory repository classes, nothing external touched.
`"supabase"`	`SupabaseStorageProvider`	Requires `DATABASE_URL` ; throws immediately if missing, rather than constructing a pool that would only fail on first use.
`"postgres"`	throws `"Postgres storage provider not implemented."`	Reserved name, not built.
`"sqlite"`	throws `"SQLite storage provider not implemented."`	Reserved name, not built.

`StorageFactory.createFromEnvironment()` is the real entry point every caller actually uses. It has one hardcoded override: under `NODE_ENV=test` , it always returns `MemoryStorageProvider` , regardless of what `PARMANA_STORAGE` is set to. This exists because whatever `PARMANA_STORAGE` named as a pure import-time side effect, which crashed test collection with supabase-js's generic `"supabaseUrl is required."` the moment `SUPABASE_*` was unset, regardless of what any individual test actually needed.

Lazy construction, everywhere that matters

`packages/api/src/repositories.ts` wraps every repository export in a `Proxy` via a local `lazyRepository()` helper, so importing `repositories.ts` (which every route file transitively does) never itself constructs a live

Supabase client. Only the first actual method call on a repository triggers `StorageFactory.createFromEnvironment()`.

The first version of `packages/api/src/application.ts`'s new `policyRepository` export constructed `SupabasePolicyRepository(PostgresPoolFactory.create())` directly at module scope:

```
export const policyRepository: PolicyRepository =
  process.env.NODE_ENV !== "test" && config.storage.provider !== "memory"
    ? new SupabasePolicyRepository(PostgresPoolFactory.create())
    : new FilePolicyRepository(config.policy.directory);
```

Because `PostgresPoolFactory.create()` is a process-wide singleton (see below), this opened `application.ts` was first imported, before any caller had a chance to configure it differently. This broke `examples/tutorials/89-readiness-probe/run.ts`'s own "genuinely unreachable database" scenario: the tutorial tries to swap in a bad `DATABASE_URL` for one specific test case, but the pool had already been created against the real, working one by the time that scenario ran, so the readiness check kept reporting `READY` when it should have reported `NOT_READY`. The fix (`packages/api/src/application.ts`, current version) wraps `policyRepository` in the identical `Proxy`-based lazy pattern `repositories.ts` already uses, deferring construction to the first real `load()` / `save()` / `listAll()` call.

Direct Postgres, not PostgREST

`PostgresPoolFactory` (`packages/storage/src/postgres/PostgresPoolFactory.ts`) is a lazy, process-wide singleton `pg.Pool`. Every Supabase-backed repository in this codebase connects through it directly, not through `supabase-js`'s REST-based client (PostgREST). The class replaced it everywhere it was still used. This removes an entire layer (PostgREST) from the own history names explicitly (see `SupabaseCallerAuditSink`'s own comments for the originating case).

As of 2026-09-16, `PostgresPoolFactory.create()`'s `Pool` is constructed with `connectionTimeoutMillis: 5000`. Before that, an unreachable or misconfigured `DATABASE_URL` hung a connecting query indefinitely rather than failing fast, found via the same readiness-probe tutorial above, whose "genuinely unreachable database" scenario never returned before this was added.

The rate-limit store, a Postgres-backed class with no repository interface

`PostgresRateLimitStore` (`packages/storage/src/postgres/PostgresRateLimitStore.ts`) is not part of `StorageProvider`, it implements `express-rate-limit`'s own `Store` contract directly (structurally, via a locally-defined `RateLimitStoreLike` interface, so `@parmana/storage` never depends on the `express-rate-limit` package itself). It exists to close a fleet-wide gap: `express-rate-limit`'s default `MemoryStore` counts per-process, so on a horizontally-scaled deployment the effective ceiling for a caller becomes `limitPerMinute * machineCount` rather than the configured limit. See Chapter 16 for the field exists to resolve.

An orphaned repository, honestly

`packages/storage/src/memory/MemoryPolicyRepository.ts` exists, is exported from `@parmana/storage`'s public index, and implements `PolicyRepository`. As of this writing, it is not referenced anywhere in `MemoryStorageProvider`, `StorageProvider`, or `application.ts`, the actual in-memory policy path uses `FilePolicyRepository` against a scratch or configured directory instead. This is the same category of finding as `@parmana/replay`: real, presumably tested code that is not part of the live wiring. Confirm this yourself before relying on it (see "How to Validate" below), it may simply be unused, or it may be a recent addition not yet wired in.

The schema itself

`supabase/migrations/*.sql` is the real source of truth for every table these repositories read and write. As one directly-verified example, the `policies` table added 2026-09-16 (`supabase/migrations/20260916060000_add_policies_table.sql`) is a two-column-key table:

```
CREATE TABLE IF NOT EXISTS policies (
    policy_name TEXT NOT NULL,
    policy_version TEXT NOT NULL,
    content_json JSONB NOT NULL,
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (policy_name, policy_version)
);
```

Note this table is read and written by `SupabasePolicyRepository` (`packages/policy/src/SupabasePolicyRepository.ts`), which lives in the `@parmana/policy` package, not `@parmana/storage`, it is constructed directly in `application.ts`, not exposed through `StorageProvider`. This is a real architectural asymmetry: every other Postgres-backed repository in this codebase goes through the `StorageProvider` facade; policy content does not. Neither the code nor any comment currently explains why this one repository sits outside the facade rather than being added to it as a sixth field, treat this as an open question rather than an assumed design decision.

The Database Schema Reference is a detailed, table by table reference covering every table in this schema; this chapter verified one table directly against the migration SQL rather than only citing that page, and recommends the same discipline for any other table you need to rely on.

How It Enables Things, With a Concrete Example

`examples/tutorials/116-supabase-policy-repository/run.ts` exercises `SupabasePolicyRepository` directly against a minimal fake `pg.Pool` (no real network), proving `save()` then `load()` round-trips exactly, `load()` on a missing entry throws `PolicyNotFoundError`, and `listAll()` reports what was saved, the same contract `FilePolicyRepository` upholds, just backed by a table instead of the filesystem.

`examples/tutorials/115-per-limiter-rate-limit-stores/run.ts` exercises `PostgresRateLimitStore` against a similar fake `pg.Pool`, proving the real `ERR_ERL_STORE_REUSE` failure mode and its fix (Chapter 16 covers this fully).

How to Validate This Yourself

- `packages/storage/src/StorageProvider.ts`, `StorageFactory.ts`, the interface and the selection logic.
- `packages/storage/src/memory/MemoryStorageProvider.ts`, `packages/storage/src/supabase/SupabaseStorageProvider.ts`, the two real implementations.
- `packages/storage/src/postgres/PostgresPoolFactory.ts`, the shared connection pool, its
- `packages/api/src/repositories.ts` and `packages/api/src/application.ts`, the lazy construction pattern, and the place it was once violated.
- `supabase/migrations/*.sql`, the real, current schema. Read the actual files; a reference document can drift, the migration files cannot (they are the thing that ran).
- `packages/storage/tests/unit/postgres-pool-factory.test.ts`, confirms the exact `Pool` constructor arguments, including `connectionTimeoutMillis`.
- Search the codebase yourself for `MemoryPolicyRepository` usage before trusting this chapter's claim that it is orphaned; that kind of claim is exactly the kind that goes stale fastest.

Integration Requirements

Variable	Required when	Effect
`PARMANA_STORAGE`	Always (defaults to `memory`)	`memory` or `supabase`; selects the `StorageProvider`.
`DATABASE_URL`	`PARMANA_STORAGE=supabase`	Direct Postgres connection string. `StorageFactory.create()` throws immediately if this is missing under `supabase`, naming both knobs, rather than deferring to a confusing `PostgresPoolFactory` error later.
`NODE_ENV=test`	Test runs	Forces `MemoryStorageProvider` regardless of `PARMANA_STORAGE`.

Chapter 14: The API and HTTP Boundary

What It Is

`packages/api/src/app.ts`'s `createApp()` is the single function that assembles every HTTP route this system exposes into one Express application, in a specific, deliberate mounting order. Everything an external caller can reach, from an unauthenticated health check to a signed execution request, passes through this one file's wiring.

Why It Was Built

An HTTP API needs a consistent, auditable answer to two questions for every route: does this route require a caller identity, and what happens when something goes wrong. `app.ts` answers both in one place rather than leaving each route handler to decide independently, which is what makes it possible to state precisely (not just claim) which routes are public and why.

How It Works

Mounting order, and why it matters

Routes are mounted in this order, and the order is load-bearing: caller-auth middleware (`createCallerAuthMiddleware`) is only added to the pipeline partway through, so everything mounted before it is unauthenticated by construction, not by an exception carved out of a uniform rule.

Path	Auth required	Why
`GET /health`	No	Liveness probe; a PaaS orchestrator has no API key to present.
`GET /ready`	No	Readiness probe (see Chapter 19); same reasoning as `/health`.
`GET /openapi.yaml`, `/openapi.json`	No	A caller cannot discover how to get a key from a spec it is not allowed to read.
`GET /api-manifest.json`	No	Self-description of SDK versions, same public-discovery reasoning.
`/documentation`, `/reference`	No	API documentation consumers.
`POST /refusal/verify`	No	: unauthenticated, third-party signature verification, not a data-access route.
`POST /audit/verify`	No	Same category as above, over `caller_audit_events` instead of Refusal Records, pure signature-over-bytes, no database lookup.
`/keys/:keyId`, `/.well-known/jwks.json`	No	Public-key discovery (PQC audit RED-2); a third party verifying a signature cannot be required to already hold a credential.
, caller-auth middleware mounted here if `callerAuth !== "disabled"`,		
`GET /`	No (mounted before, but trivial)	Returns `{ name: "Parmana", status: "UP" }`.
`/version`	Yes (if auth enabled)	
`/callers/me`	Yes	Caller self-lookup: identity and exactly what it's authorized to do.
`/execute`	Yes	Rate-limited by `callerId` (Chapter 16); the limiter itself is only mounted when caller-auth is enabled, since there is no caller identity to key off otherwise.
`/verify`, `/verification`	Yes	
`/refusal`	Yes	Ownership-scoped lookup by ID (the unauthenticated verify route above is separate).
`/receipt`, `/receipt/latest`	Yes	
`/transactions`	Yes	

Path	Auth required	Why
`/policies`	Yes	Mounted twice: once for the policy-content routes (`policies.ts`), once for the Policy Governance pending-changes routes (`pending-policy-changes.ts`), their path shapes don't collide.
`/trust-records`	Yes	
`/replay`	Yes	
, error handler mounted last,		

(`packages/api/src/app.ts`)

Error mapping

`createErrorHandler()` (`packages/api/src/middleware/error-handler.ts`) is the single place a thrown domain error becomes an HTTP response:

Thrown	Status	Notes
Malformed/oversized JSON body (`express.json()` , before any route handler)	400 / 413	Best-effort audited even though no caller identity exists yet, deliberately fail-**open** here, unlike every other caller-audit write, since the correct rejection has already happened by the time this fires.
`BusinessTransactionValidationError` , `PolicyValidationError` , `SignalValidationError`	400	
`PolicyNotFoundError`	404	
`DuplicateBusinessTransactionError`	409	
`NonceAlreadyConsumedError`	`error.status` (its own code)	Distinguished from a generic `RuntimeError` so a caller can tell "this already ran" apart from "something broke" without string-matching a 500 body.
Any `RuntimeError`	`error.status`	
Anything else	500	Logged via `console.error` , response body is the generic `"Internal Server Error"` , no internal detail leaked.

(`packages/api/src/middleware/error-handler.ts:95-201`)

The full route inventory

Every route module lives in `packages/api/src/routes/` :

File	Mounted at	Purpose
`health.ts`	`/health`	Liveness.
`ready.ts`	`/ready`	Readiness (Chapter 19).
`openapi.ts` / `openapi-json.ts`	`/openapi.yaml` / `/openapi.json`	Machine-readable API spec.
`api-manifest.ts`	`/api-manifest.json`	SDK version self-description.
`documentation.ts` / `reference.ts`	`/documentation` / `/reference`	Human-readable docs.
`refusal-verify.ts`	`/refusal/verify`	Public signature verification of a Refusal Record.
`audit-verify.ts`	`/audit/verify`	Public signature verification of an audit event.
`keys.ts`	`/keys/:keyId` , `/.well-known/jwks.json`	Public key discovery.
`callers-me.ts`	`/callers/me`	Caller self-lookup.
`execute.ts`	`/execute`	Submit and execute a Business Transaction.

File	Mounted at	Purpose
`verify.ts` / `verify-get.ts`	`/verify` / `/verification`	Verification of an executed transaction.
`refusal-get.ts`	`/refusal`	Ownership-scoped Refusal Record lookup.
`receipt.ts` / `receipt-get.ts`	`/receipt` / `/receipt/latest`	Receipt issuance and lookup.
`transactions.ts`	`/transactions`	Business Transaction create-and-execute (parity with `/execute`).
`policies.ts`	`/policies`	Policy content read/validate routes.
`pending-policy-changes.ts`	`/policies`	Maker-checker propose/list/approve/reject (Chapter 7).
`trust-records.ts`	`/trust-records`	Bulk export.
`replay.ts`	`/replay`	Replay a prior execution.
`version.ts`	`/version`	
`findOpenApiSpecFile.ts`	(helper, not a route)	Locates the built OpenAPI spec file on disk.

`trust proxy`

`app.set("trust proxy", 1)` (`packages/api/src/app.ts:147`) trusts exactly one proxy hop, this codebase's actual deployment fronting (Fly.io's edge, per `fly.toml`'s `force_https`). Without this, `req.ip` and `req.protocol` / `req.secure` reflect the proxy's own connection to the process, not the original client's.

How It Enables Things, With a Concrete Example

`examples/tutorials/102-distinguishable-http-status` demonstrates the `NonceAlreadyConsumedError` vs. generic-`RuntimeError` distinction in the error handler directly, a policy denial and a replay attempt against the same endpoint return genuinely different, distinguishable status codes rather than both collapsing into an opaque failure.

How to Validate This Yourself

- `packages/api/src/app.ts`, the actual mounting order; read it top to bottom rather than trusting the table above once this file changes.
- `packages/api/src/middleware/error-handler.ts`, the full error-to-status mapping.
- `packages/api/src/routes/*.ts`, one file per route group.
- `packages/api/tests/integration/*.integration.test.ts`, most route groups have a corresponding integration test that exercises the real HTTP path, not just the handler function in isolation.

Integration Requirements

`createApp()` requires a `CallerAuthOption`, either a real `{ authenticator, auditSink }` pair or the literal string `"disabled"` (used only for local development and the tutorial suite; there is deliberately no default, so omitting this choice is not possible). Optional: `RateLimitOption` (Chapter 16), `stepUpVerifier` and `policyChangeApprovalService` (both required in effect, though optional at the type level, once caller-auth is enabled and the Policy Governance approve/reject endpoints are actually reachable, see Chapter 7).

Chapter 15: Caller Authentication and Scoping

What It Is

The layer that decides whether an HTTP request is entertained at all, who it came from, and what that identity is actually allowed to do (which principal it may assert, which capabilities it may invoke). It runs before a Business Transaction is even constructed and is entirely independent of policy evaluation and gateway attestation, which run later and answer different questions.

Why It Was Built

Without an identity layer, authority/authorization fields on a submitted transaction are purely caller-declared and never cross-checked against who is actually calling, any caller holding any valid API key could claim to be any human or role in the resulting signed trust record. Caller authentication and scoping close that gap at the door, before anything downstream has a chance to trust a claim it shouldn't.

How It Works

Authentication: `StaticKeyAuthenticator`

`packages/api/src/auth/StaticKeyAuthenticator.ts` authenticates against a static, pre-hashed set of API keys (`ApiKeyEntry[]`, from `PARMANA_API_KEYS`). Keys are never held in plaintext, only a SHA-256 hash of each is compared, via `timingSafeEqual` against the hash bytes (never the raw key), so a leaked config never yields a usable secret and comparison timing never leaks information about a partial match.

Multiple entries may share the same `callerId`, this is how key rotation works: add the new key's hash, keep the old one active during migration, then remove the old entry to revoke it, with no downtime and no code change.

`ApiKeyEntry` (`packages/shared/src/config/ApiKeyEntry.ts`):

```
export interface ApiKeyEntry {
  readonly callerId: string;
  readonly keyHash: string;
  readonly credentialHolderType?: AuthorityType;
  readonly allowedPrincipalIds?: readonly string[];
  readonly allowedCapabilities?: readonly string[];
  readonly stepUpPublicKey?: string;
}
```

`credentialHolderType` is operator-declared metadata set only at issuance time, distinct from `Authority.authorityType`, which is caller-declared and lives on the Business Transaction itself describing the business action's asserted authority. The two must not be conflated: one describes who was handed the credential, the other describes what a specific request claims. `isHumanCaller.ts` is the only consumer of `credentialHolderType`, and treats every value other than exactly `AuthorityType.USER`, including `undefined`, as non-human, fail-closed by default.

The middleware itself

`createCallerAuthMiddleware()` (`packages/api/src/middleware/caller-auth.ts`) extracts a bearer token, authenticates it, and on success attaches `callerId`, `callerAllowedPrincipalIds`, `callerAllowedCapabilities`, `callerCredentialHolderType`, and `callerStepUpPublicKey` onto the Express `Request` object (a local type augmentation, same pattern `@parmana/envelope-verifier`'s own Express typing uses). On failure it returns 401 with a `WWW-Authenticate: Bearer` header, and either way it records a `caller.rejected` or `caller.authenticated` audit event through `recordCallerAuditEvent` before proceeding, **fail-closed**: if the audit write itself fails, the request is rejected with `AuditUnavailableError` (503) rather than proceeding unaudited. This applies to both outcomes, not just denials, a perfectly valid credential whose `caller.authenticated` write fails also gets 503, not the 200 it would otherwise get.

Scoping: principal and capability

Two independent checks, deliberately opposite in their fail-closed default:

`isPrincipalAllowed()` (`packages/api/src/auth/isPrincipalAllowed.ts`) decides whether a caller may assert a given `authority.principalId` . Default (no `allowedPrincipalIds` configured): a key may only assert **itself**, `principalId` must equal `callerId` exactly. An unconfigured key proves its own identity and nothing more.

`isCapabilityAllowed()` (`packages/api/src/auth/isCapabilityAllowed.ts`) decides whether a caller may invoke a given capability (the transaction's `intent.action`) at all. Default (no `allowedCapabilities` , or an empty list): **every** capability is denied. There is no meaningful "may invoke its own capability" fallback the way "may only assert itself" is for principals, an unconfigured key is authorized to invoke nothing until explicitly granted. The literal string `"*"` in `allowedCapabilities` is an explicit, auditable wildcard, never an implicit default.

`PARMANA_AUTH_DISABLED`

A deployment can set `PARMANA_AUTH_DISABLED=true` to accept every request with no caller authentication at all. This is surfaced not just as a startup log line but as a field on `GET /ready` 's own response body (`packages/api/src/routes/ready.ts:44-52`):

```
{
  "authDisabled": true,
  "warning": "PARMANA_AUTH_DISABLED=true -- this deployment is accepting requests with no caller authen
}
```

This exists because a log line is easy to miss after the fact in a log-aggregation tool; a field on the readiness probe every PaaS orchestrator already polls every 30 seconds is something an operator's own monitoring can assert and alert on directly.

How It Enables Things, With a Concrete Example

`examples/tutorials/101-fail-closed-caller-audit-writes/run.ts` proves the audit-write fail-closed guarantee directly: it simulates the storage outage `SupabaseCallerAuditSink` would surface, and shows both `caller.rejected` and `caller.authenticated` outcomes react identically (503, not their otherwise-normal status) when the audit write itself fails. There is no retry, buffering, or queueing, a failure fails closed immediately, once, per request.

`packages/api/tests/integration/caller-scoping.integration.test.ts` , `caller-principal-scoping.integration.test.ts` , and `caller-capability-scoping.integration.test.ts` exercise the two scoping checks at the real another's transactions, receipts, trust records, and refusal records by ID).

How to Validate This Yourself

- `packages/api/src/middleware/caller-auth.ts` , the middleware itself.
- `packages/api/src/auth/StaticKeyAuthenticator.ts` , `hashApiKey.ts` , authentication.
- `packages/api/src/auth/isPrincipalAllowed.ts` , `isCapabilityAllowed.ts` , `isHumanCaller.ts` , `isOwnedByCaller.ts` , the four scoping/identity predicates.
- `packages/shared/src/config/ApiKeyEntry.ts` , the credential shape.
- `packages/api/src/routes/ready.ts` , the `authDisabled` warning field.
- `packages/api/tests/integration/caller-*.integration.test.ts` , the real HTTP-level proof for every claim above.

Integration Requirements

Variable	Effect
`PARMANA_API_KEYS`	JSON array of `ApiKeyEntry` objects, the entire credential set for a deployment.
`PARMANA_AUTH_DISABLED`	`true` disables caller-auth entirely. Never set this in a real deployment; the `/ready` endpoint will say so loudly if you do.

Provisioning a new credential: `scripts/generate-api-key.ts` (see Chapter 7 for the step-up-keypair variant used by Policy Governance checkers).

Chapter 16: Rate Limiting

What It Is

Three independent `express-rate-limit` limiters, one for `POST /execute` (keyed by authenticated caller identity), one for `GET /health` / `GET /ready` (keyed by IP, far more signatures or write data (`POST /refusal/verify` , `POST /execution-intents/verify` , `POST /audit/verify` , `/handbook` ; keyed by IP, 60 per minute by default). Each is backed by either an in process store or a durable, fleet wide Postgres store depending on deployment configuration.

Why It Was Built

Two separate problems. First, ordinary abuse/overload protection on the one endpoint that actually does real work (signing, storage writes). Second, a real production-readiness gap horizontally-scaled deployment the effective ceiling for a caller becomes `limitPerMinute * machineCount` rather than the configured, intended limit. `PostgresRateLimitStore` exists to close that gap for any deployment where it matters.

How It Works

Two limiters, deliberately different keying

`createExecuteRateLimiter()` (`packages/api/src/middleware/rate-limit.ts:71-90`) keys by `req.callerId` , not IP, a design-partner integration commonly calls from a shared backend IP, where an IP-keyed limit would either starve every caller behind it or be too loose to mean anything. It is only ever mounted when caller authentication is enabled (`app.ts`), since `req.callerId` does not exist otherwise, and it runs ahead of the route handler, so a rejected request never reaches policy evaluation, signing, or storage, nothing is consumed for a request this middleware rejects.

`createHealthReadyRateLimiter()` (`:103-115`) keys by IP (the package default, IPv6-safe in `express-rate-limit` v8), and is deliberately far more permissive, these routes are cheap and legitimately polled on a fixed interval by PaaS health-check infrastructure (`fly.toml` 's own check runs every 30 seconds per machine); this limiter must never be tight enough to throttle that.

Both share one 429 response shape via `rateLimitHandler()` , which sets `Retry-After` explicitly from the store's own `resetTime` rather than trusting a package default header.

One shared store, two limiters

An earlier version constructed a single `PostgresRateLimitStore` and passed it to both limiters. `express-rate-limit` v8's own documented contract disallows this, a `Store` instance must not back more than one limiter, and the library throws `ERR_ERL_STORE_REUSE` the moment a second limiter's `init()` runs against an already-initialized store. Passing one shared instance to both `createExecuteRateLimiter` and `createHealthReadyRateLimiter` crashed every request on a fresh cold start.

A documented correction, not silently fixed: it was first assumed this crashed the whole Reading `express-rate-limit` 's actual installed source shows every validation is wrapped in a try/catch that only logs the violation and never re-throws, the reuse warning is real, but harmless on its own. The 500 that request actually returned was caused by a completely The fix (two separate `Store` instances, each with a distinct `prefix`) is still correct and worth having, independent of the corrected causal story, it is the library's own documented usage contract, and a stricter `validate` config in a future version could make this fatal for real.

The fix itself: `RateLimitOption` (`packages/api/src/app.ts`) carries two distinct fields, `executeStore` and `healthStore` , never one shared `store` . `createRateLimitStore()` (`packages/api/src/bootstrap/createRateLimitStore.ts`) is called once per limiter, each with its own `prefix` (e.g. `"execute:"` and `"health:"`), so their counters can never collide in the shared `rate_limit_counters` table even though both instances share the same underlying `Pool` (`PostgresPoolFactory.create()` is itself a singleton, so calling this twice does not open a second connection pool).

`PostgresRateLimitStore.prefix` (`packages/storage/src/postgres/PostgresRateLimitStore.ts:80`) is deliberately **public**, not a private implementation detail, this is `express-rate-limit` 's own documented `Store.prefix` contract,

and naming the field exactly `prefix` is what lets the library's own reuse/double-count validation recognize two instances with different prefixes as legitimately distinct.

Three-way store selection

`createRateLimitStore(prefix)` returns:

Condition	Returns
`NODE_ENV=test`	`undefined`, falls back to `express-rate-limit`'s own in-process `MemoryStore`.
No `DATABASE_URL`	`undefined`, with a loud `console.warn` naming the exact tradeoff. Deliberately **not** fail-closed.
`DATABASE_URL` configured	`new PostgresRateLimitStore(PostgresPoolFactory.create(), prefix)`, durable, fleet-wide.

The "no `DATABASE_URL`" case is deliberately not fail-closed the way this codebase's nonce store (`createNonceStore.ts`) is: an in-process rate limiter is a real, working control on a single instance (this deployment's actual current shape, `fly.toml` pins `min_machines_running = 1`), just not fleet-wide accurate the moment a second machine joins. A missing nonce store means replay protection silently vanishes, a security bypass; a missing shared rate-limit store means the ceiling is merely looser than configured, not absent. Refusing to start over that would break every single-instance and local deployment that works correctly today.

How It Enables Things, With a Concrete Example

`examples/tutorials/115-per-limiter-rate-limit-stores/run.ts` reproduces the real `ERR_ERL_STORE_REUSE` warning directly (Scenario 1: one shared store, logged but not thrown) and confirms the fix (Scenario 2: two prefixed stores, no warning at all), hermetically, against a minimal fake `pg.Pool`, no real database needed. Its own README documents the corrected diagnosis explicitly, kept visible rather than quietly rewritten.

`packages/api/tests/integration/rate-limit.integration.test.ts` exercises both limiters at the real HTTP boundary: normal traffic passing through, a rate-limited caller not blocking a different caller, a 429 with a correct `Retry-After` header and no execution side effects, and confirms the limiter is entirely skipped when caller-auth is disabled.

How to Validate This Yourself

- `packages/api/src/middleware/rate-limit.ts`, both limiter factories.
- `packages/api/src/bootstrap/createRateLimitStore.ts`, the three-way selection logic, with its own comment stating the exact tradeoff for each branch.
- `packages/storage/src/postgres/PostgresRateLimitStore.ts`, the durable store, including the `prefix` field's own doc comment explaining exactly why it must be named that and be public.
- `packages/api/src/app.ts, 149-171, 248-264`, where both limiters are actually constructed and mounted.

Integration Requirements

Variable	Effect
`RATE_LIMIT_EXECUTE_PER_MINUTE`	Overrides the default execute limiter ceiling (`DEFAULT_EXECUTE_PER_MINUTE = 30`).
`RATE_LIMIT_HEALTH_PER_MINUTE`	Overrides the default health and ready limiter ceiling (`DEFAULT_HEALTH_PER_MINUTE = 300`).
`RATE_LIMIT_PUBLIC_PER_MINUTE`	Overrides the default ceiling for the unauthenticated verify routes, `/handbook` and key discovery (`DEFAULT_PUBLIC_PER_MINUTE = 60`).
`RATE_LIMIT_AUTH_FAILURE_PER_MINUTE`	Overrides the default ceiling for failed caller authentications per IP (`DEFAULT_AUTH_FAILURE_PER_MINUTE = 30`). Only `401` responses count.
`DATABASE_URL`	If set, every limiter uses the durable, fleet wide `PostgresRateLimitStore` instead of per process memory.

Chapter 17: Audit and Evidence Trails

What it is

Parmana keeps several distinct, signed, durable trails of what happened: refusal records for rejected policy decisions, caller audit events for every authentication and authorization decision, execution audit events for the execution control lifecycle, and an explicit evidence anchor that ties a trust record's policy content, governance status, and connector evidence together into one checkable pointer. These are separate mechanisms with separate scopes, not one generic "audit log," and each exists to answer a specific question an auditor might ask.

Why it was built

An approval trail is only half the story. A system that only records what it approved cannot prove it correctly rejected everything it should have rejected, cannot prove a caller's authentication decisions were consistent over time, and cannot prove that a trust record's policy content, its governance status, and the connector evidence backing it are genuinely linked rather than merely sitting next to each other in the same object. Each trail here closes

How it works

Refusal Records

`RefusalRecord` (`packages/shared/src/domain/refusal-record.ts`) is the durable, signed, independently verifiable REJECT-path counterpart to `ExecutionTrustRecord`. Its own doc comment is explicit about scope: it covers `PolicyEngine.evaluate` REJECTs and `SignalIntentBinder` binding-violation REJECTs only, not every kind of rejection this system can produce. Caller-auth failures and webhook signature failures are a separate, unsigned audit-sink milestone (see Unlike `ExecutionTrustRecord`, a `RefusalRecord` is not an append-only aggregate with sub-histories: a refusal is a single terminal event, so there is at most one per `businessTransactionId`.

The record carries the exact rejected `Decision` (not summarized or reconstructed, the same `Decision` object `RuntimeEngine` already built), the `evaluatedIntent` snapshot, and, only when the rejection came from `SignalIntentBinder`, `bindingViolations`. Since 2026-10-06 it also carries `policyContentHash`, the hash of the policy content that decided the refusal, computed by `RuntimeEngine` from the policy that loaded, the same value an `ExecutionTrustRecord` carries in `transaction.policy.contentHash`; older records omit it and still verify. `RefusalRecordBuilder` (`packages/runtime/src/RefusalRecordBuilder.ts`) constructs and hashes the draft, then `RefusalCrypto` (`packages/crypto/src/RefusalCrypto.ts`) signs it, deliberately reusing the same signing stack and `DEFAULT_KEY_ID` as `VerificationCrypto`, so approvals and refusals share one root of trust, not two keys to manage. `RefusalCrypto.canonicalRecord()` excludes the signature itself from what gets hashed and signed, the same discipline every other signed artifact in this codebase follows.

Caller Audit Events

`SupabaseCallerAuditSink` (`packages/api/src/auth/SupabaseCallerAuditSink.ts`) durably records every caller-authentication decision: success, failure, capability checks, principal checks. sink lost every event on restart), and four of its columns were each added in a separate, dated migration as a distinct milestone landed, `capability` (2026-08-12, capability-scoping), `principalId` (2026-08-16, principal-scoping), `severity` (2026-08-18, policy-governance), Every event is signed at write time (`AuditEventCrypto`), before any storage-only field like an insert timestamp is added, a plain durable row could otherwise be altered by anyone with direct database access with no way to detect it.

One real, named operational detail worth knowing if you ever touch this class: it writes via a direct Postgres connection (`PostgresPoolFactory`), not `supabase-js`, because of a confirmed `signature_json` column. This is documented in the class's own comment as a temporary workaround at the PostgREST layer specifically, not a problem with the database or this codebase's own schema.

Failure semantics are explicit and deliberately unchanged from the sink this replaced: `record()` is an unguarded `await` in the caller-auth middleware with no try/catch around it. A write failure here rejects the promise exactly like any other failed Supabase insert elsewhere in this codebase; it is the caller's existing (unmodified) behavior that decides what happens next, documented rather than silently changed.

Execution Audit Events

`SupabaseExecutionAuditSink` (`packages/storage/src/supabase/SupabaseExecutionAuditSink.ts`) covers the execution-control lifecycle separately from caller authentication. Per the Database Schema Reference (verify this claim yourself if you touch this table, it is a real operational fact worth confirming against current deployment config, not just trusting a comment), the underlying `execution_audit_events` table is shared cross-repository with a separate `example-agent` repository, meaning this table's schema is a contract between two codebases, not something to change unilaterally from this one.

Evidence Anchor

`EvidenceAnchor` (`packages/shared/src/domain/evidence-anchor.ts`) is the newest of these mechanisms. Its own doc comment states plainly what it is and is not: not a new cryptographic guarantee (`policyContentHash`, the governance anchor, and connector evidence were already bound together implicitly, since all three already sit inside the same `ExecutionTrustRecord` that `trustRecordHash` / `signature` cover in full), what it adds is a single, explicitly-named, independently-computed pointer, `{policyContentHash, governanceAnchorStatus, connectorEvidenceHash, anchorHash}`, that an auditor can check without already knowing to reach into `transaction.policy` and `executions[].evidence.attributes.connector` separately and reconstruct the binding themselves. `anchorHash` is a `TrustRecordHasher` hash of the canonicalized triple, computed the same way every other artifact hash in this codebase is. `BusinessTrustRecordBuilder.buildEvidenceAnchor()` (`packages/runtime/src/`) builds it; every field is optional individually (absent when there was nothing to anchor), but `anchorHash` is always present.

Policy Governance Anchor Resolver

`PolicyGovernanceAnchorResolver` (`packages/api/src/governance/PolicyGovernanceAnchorResolver.ts`) performs three checks against `PolicyChangeApprovalRecordRepository`, in order: does an approval record exist for this `(policyName, policyVersion)` at all (`NO_APPROVAL_RECORD` if not); does its signature verify (`SIGNATURE_INVALID` if not); does its `contentHashAfter` match the policy content actually used for this decision (`CONTENT_MISMATCH` if not). A clean pass returns `VERIFIED`. Critically, this class is wired unconditionally into `RuntimeEngine`, with no feature flag, because it only ever records what it found, it never blocks execution. This is the deliberate opposite of `PolicyGovernanceExecutionVerifier` (the execution-time enforcement gate covered in Chapter 8/10's territory, not here), which has the same three checks but is enforced (on everywhere except `NODE_ENV` `test` and `development`) precisely because a violation there must refuse a real execution. The class's own comment names this distinction explicitly and traces it to evidentiary concerns in separate types even when their logic overlaps, rather than deduplicating into one shared helper with two different blast radii.

How it enables things, with a concrete example

`packages/api/tests/integration/refusal-record.integration.test.ts` and `packages/api/tests/integration/structural-validation-audit.integration.test.ts` exercise the refusal and caller-audit paths at the real HTTP boundary. No standalone tutorial in `examples/tutorials/` exercises `EvidenceAnchor` / `PolicyGovernanceAnchorResolver` directly as of this writing, their coverage lives in `packages/api/tests/unit/PolicyGovernanceAnchorResolver.test.ts` and two `packages/runtime/tests/e2e/runtime.e2e.test.ts` cases (result stamped correctly on a real execution; a resolver failure never blocks one). Said honestly: this is real, tested, production-wired code, just not yet demonstrated as a narrative tutorial the way policy governance itself is (Chapter 7).

How to validate this yourself

- `packages/shared/src/domain/refusal-record.ts`, `packages/runtime/src/RefusalRecordBuilder.ts`, `packages/crypto/src/RefusalCrypto.ts`
- `packages/api/src/auth/SupabaseCallerAuditSink.ts` and its migrations, named in its own comment, under `supabase/migrations/`
- `packages/storage/src/supabase/SupabaseExecutionAuditSink.ts`
- `packages/shared/src/domain/evidence-anchor.ts`, `packages/api/src/governance/PolicyGovernanceAnchorResolver.ts`
- `packages/api/tests/unit/PolicyGovernanceAnchorResolver.test.ts`, `packages/api/tests/integration/refusal-record.integration.test.ts`, `packages/api/tests/integration/structural-validation-audit.integration.test.ts`

Integration requirements

`DATABASE_URL` (direct Postgres connection) for `SupabaseCallerAuditSink` and `SupabaseExecutionAuditSink` to persist durably; without it, this codebase falls back to in-memory sinks that lose events on restart (correct for tests, not for production). No separate configuration is needed for `EvidenceAnchor` / `PolicyGovernanceAnchorResolver`, they are wired unconditionally whenever `RuntimeEngine` is constructed.

Chapter 18: Independent Verification

What it is

A third party with no access to a running Parmana instance at all, no API key, no network access to this deployment, should still be able to verify that a signed trust record, receipt, or refusal record is genuine and unmodified, given only the artifact itself and the relevant public key. This chapter covers the offline verification path, how a public key is discovered in the first place, and an honest assessment of two packages that sound like they belong here by name but turned out not to both be wired the way their names suggest.

Why it was built

Every signature this codebase produces is meant to be checkable without trusting Parmana's own server to tell you it's valid, that would defeat the purpose of signing anything. Offline verification exists so the check can be performed with zero network calls, zero disk reads beyond the artifact and a public key you already have, and zero environment configuration.

How it works

Offline verification

`verifyExecutionTrustRecordOffline()` (`packages/crypto/src/OfflineVerifier.ts`) is explicitly built to have zero external dependencies at call time: the caller supplies the exact public key material for every `keyId` a record references, and everything else, canonical serialization, the exact field mapping the signature actually covers, the Ed25519/ML-DSA-65 signature implementations, is the same code the real, online `VerificationCrypto` uses. `ExecutionTrustRecordCanonicalView.ts` is shared between the offline and online paths specifically so there is exactly one definition of "which fields participate in the signature," not two implementations that could silently drift apart from each other.

It checks three things, and reports each independently rather than collapsing them into one opaque boolean: `hashValid` (does the record's own `trustRecordHash` match a fresh hash of its canonical content), `legacySignatureValid` (does the always-present legacy `signature` field verify), and `hybridSignaturesValid` (only present when the record actually carries a `signatures` array, a hybrid record needs at least two distinct-algorithm entries, and a duplicate algorithm in that array is treated as invalid). The function's own comment is explicit about a real limitation: the hash check recomputes SHA-256 unconditionally, because that's the only hash algorithm this codebase actually implements today (other config-declared algorithms like `sha3-512` / `blake3` have no real provider), a record hashed under a genuinely different algorithm could not be distinguished from a tampered one by this check alone, since the algorithm used isn't itself recorded on the artifact separately from the hash value.

The offline verifier is a separate, public repository, `pavancharak/parmana-verifier`. It recomputes a record's hash and checks its signatures against keys the verifier operator pins. It makes no network calls.

Large records signed under AWS KMS

A record over 4096 bytes of canonical content is signed by the KMS signer as a fixed 97 byte commitment (the prefix `PARMANA-ED25519-LARGE-MESSAGE-V1`, a NUL byte, and the SHA-512 digest of the canonical bytes), because KMS refuses a longer raw message. The signature is still Ed25519. An independent verifier must accept the raw signature for any record, and additionally the commitment form only for a record over 4096 bytes. A commitment signature over a smaller record must be rejected. `verifyExecutionTrustRecordOffline()` and the Python `verify_execution_trust_record_offline()` both implement this, and the cross language test in `python/tests/test_offline_verifier.py` proves the Python implementation accepts a large record TypeScript signed as a commitment. Records signed before this change were signed raw and still verify unchanged.

Public-key discovery

`createKeysRouter()` (`packages/api/src/routes/keys.ts`) exposes `GET /keys/:keyId` and `GET /.well-known/jwks.json`. Both are deliberately mounted before this app's caller-auth middleware, alongside `/refusal/verify` and `/audit/verify`, a third party cannot be required to already hold a Parmana-issued credential just to fetch the key it needs to check a signature that credential has nothing to do with. Every response includes a PEM-encoded SPKI public key (RFC 7468), which is what `OfflineVerifier.ts` and its Python

counterpart both consume directly, plus a `jwk` field when the running Node version's `KeyObject.export({format:"jwk"})` supports the key's algorithm, the comment notes ML-DSA-65 exports as JOSE/COSE key type `"AKP"` (from the IETF draft-ietf-jose-fully-specified-algorithms track, not an identifier this codebase invented) and Ed25519 as the long-standardized `"OKP"`. The JWKS endpoint is explicitly not a standards-pure RFC 7517 JWK Set, since this codebase's PEM-first shape doesn't fit that spec's structure exactly, it's a superset any consumer that only wants the `.jwk` field per entry can filter down to.

An honest assessment of `@parmana/replay` and `@parmana/receipt`

This is worth stating precisely rather than by reputation. Checking actual imports (not package existence, actual usage inside `packages/api/src`):

- **`@parmana/replay`** is a real package (`packages/replay/`), with a real `ReplayEngine`, `ReplayExecutor`, `ReplayBuilder`, `ReplayVerifier`, and a real test suite including an integration test and a determinism test. It has **zero imports anywhere inside `packages/api/src`**. It is real, tested, tutorial-capable code with no production wiring, say this plainly rather than implying it's load-bearing because its name suggests it should be.
- **`@parmana/receipt` as a standalone package, does not exist in this repository at all**, there is no `packages/receipt/` directory and no `package.json` anywhere named `@parmana/receipt`. Receipt functionality is real and genuinely wired into production, but it lives inside `@parmana/crypto` (`ReceiptCrypto.ts`, `ReceiptHasher.ts`) and the `Receipt` type in `@parmana/shared`, used directly by the runtime's trust-record construction, confirmed by real signed receipts appearing in real `ExecutionTrustRecord` output. If you have read or been told that "`@parmana/receipt` and `@parmana/replay` are both unwired," that claim is now only half true: receipts are real and live; replay is the one that remains genuinely unwired.

How it enables things, with a concrete example

`examples/tutorials/05-verification/run.ts` and `examples/tutorials/12-envelope-verification/run.ts` exercise the online verification path. `packages/crypto/tests/unit/` (search for `OfflineVerifier` or `offline-verifier`) covers the offline path directly with real, generated key material and a deliberately tampered record to confirm detection. `python/parmana/crypto/offline_verifier.py` is the Python-side equivalent of the same capability, for a consumer with no Node.js runtime at all.

How to validate this yourself

- `packages/crypto/src/OfflineVerifier.ts`, `ExecutionTrustRecordCanonicalView.ts`
- `packages/api/src/routes/keys.ts`
- `python/parmana/crypto/offline_verifier.py`
- `packages/replay/src/` and `packages/replay/tests/` (real, but check its own package.json's consumers, or the lack of any inside `packages/api/src`, to confirm the unwired claim yourself)
- `packages/crypto/src/ReceiptCrypto.ts`, `ReceiptHasher.ts` (confirm receipts are real by tracing their usage from `packages/runtime/src/` into an actual trust record)

Integration requirements

None for offline verification itself, that is the entire point, zero network calls, zero environment variables. To exercise `GET /keys/:keyId` or the JWKS endpoint against a real deployment, only `PARMANA_KEY_DIR` (or the configured `KEY_PROVIDER`) needs to be set correctly server-side; the caller needs nothing.

Chapter 19: Health and Readiness

What it is

Two distinct endpoints answer two distinct questions a deployment orchestrator needs answered separately: `GET /health` asks "is this process running at all," and `GET /ready` asks "is this process actually able to serve real traffic right now." Conflating them is a common mistake this codebase deliberately avoids.

Why it was built

A process that is up but backed by dead storage is not actually ready to serve traffic, and a PaaS orchestrator that only checks liveness has no way to route around it. `/ready` exists specifically to catch that case and let the orchestrator act on it.

How it works

`GET /health` (`packages/api/src/routes/health.ts`) is deliberately trivial: it responds `{"status": "UP"}` unconditionally, with no external check of any kind. This is a pure liveness probe, if the process can answer HTTP at all, it answers this.

`GET /ready` (`packages/api/src/routes/ready.ts`) is genuinely different: when storage is not Supabase-backed (`NODE_ENV=test`, or `PARMANA_STORAGE` set to anything other than `"supabase"`), there is no external dependency to probe, so it reports `READY` unconditionally, the same distinction `assertStorageConfigured.ts` already draws at boot. Otherwise, it actually queries Postgres with `SELECT 1` (cheap, transfers no table rows, just confirms the connection and credentials work) via `PostgresPoolFactory`, not `supabase-js`/PostgREST, the route's own comment notes this probe previously depended on PostgREST for a `consumed_nonces` HEAD request, meaning a PostgREST-layer outage could make the readiness probe itself unreliable; querying directly removes that failure mode. A failed query returns `503` with `status: "NOT_READY"` and the real error message as `reason`.

Both responses also carry an `authDisabled` field, and a `warning` string when true. This exists because `PARMANA_AUTH_DISABLED=true` is exactly the kind of misconfiguration ("copied `.env.example` without reading every line") that is easy to miss as a one-time `console.warn` at boot but easy for an operator's own monitoring or synthetic checks to catch and alert on directly when it's a field on an endpoint already polled every 30 seconds.

How it enables things, with a concrete example

`examples/tutorials/89-readiness-probe/run.ts` is the concrete example, read it directly, it is short and the three scenarios described above are exactly what it runs, printing the real status code and body for each.

How to validate this yourself

- `packages/api/src/routes/health.ts`, `packages/api/src/routes/ready.ts`
- `packages/storage/src/postgres/PostgresPoolFactory.ts` (note the `connectionTimeoutMillis`
- `packages/api/src/application.ts` (the lazy `getPolicyRepository()`/`Proxy` construction, and its own doc comment explaining why an earlier, eager version was wrong)
- `packages/api/src/repositories.ts` (`lazyRepository()`, the pattern the fix now mirrors)
- `examples/tutorials/89-readiness-probe/run.ts`
- `packages/storage/tests/unit/postgres-pool-factory.test.ts` (asserts the exact `Pool` constructor arguments, including the timeout)

Integration requirements

No additional configuration beyond what storage already requires (`PARMANA_STORAGE`, `DATABASE_URL` when Supabase-backed). A PaaS orchestrator should be configured to poll `/health` for restart decisions and `/ready` for traffic-routing decisions, treating them as interchangeable defeats the reason both exist.

Chapter 20: Integrating Parmana (Requirements and Getting Started)

What it is

Integrating Parmana means getting a caller, human, AI agent, or automated service, to submit a Business Transaction to a running Parmana instance and receive back a signed Execution Trust Record. There are three real paths to do this: the official TypeScript SDK (`typescript/`, published as `@parmana/sdk`), the official Python SDK (`python/`, published as `parmana`), or plain HTTP against the REST API directly. All three ultimately call the same two routes, `POST /execute` and `POST /transactions`, which run through the identical execution pipeline.

Why it was built this way

A caller hand-building a request body has to keep three id pairs internally consistent (`metadata.businessTransactionId` must equal the top level `businessTransactionId`, `authorization.authorityId` must equal `authority.authorityId`, `intent.authorizationId` must equal `authorization.authorizationId`). Getting any of these wrong produces a 400 from `BusinessTransactionValidator`. The SDKs exist specifically so that class of mistake cannot arise through them, not merely documented.
`typescript/src/builders/createBusinessTransaction.ts` says this directly in its own doc comment: this exact mismatch is "the class of mistake that produced every 'X must match Y' 400 response documented in the end to end Paytm guide."

How it works

The TypeScript SDK

`@parmana/sdk`'s entry point is `ParmanaClient` (`typescript/src/client/ParmanaClient.ts`). Its own doc comment is explicit about what it does and does not do: it holds configuration, composes sub-APIs, and delegates, it does not evaluate policy, authorize execution, execute business logic, verify trust records, or replay executions, all of that is server side.

Construction requires a `Configuration` (`typescript/src/config/Configuration.ts`):

```
interface Configuration {
  readonly endpoint: string; // Parmana Runtime endpoint, e.g. https://runtime.example.com
  readonly apiKey?: string; // Bearer key minted by scripts/generate-api-key.ts
  readonly timeout?: number; // default 30000ms
  readonly retryPolicy?: RetryPolicy;
  readonly transport?: Transport; // defaults to the SDK's own HTTP transport
  readonly userAgent?: string;
}
```

`apiKey` is optional only against a Runtime started with `PARMANA_AUTH_DISABLED=true`, which is local development only, every other deployment rejects an unauthenticated request with a 401 before a Business Transaction is even constructed.

`ParmanaClient` composes ten sub APIs (`HealthApi`, `ExecutionApi`, `VerificationApi`, `ReplayApi`, `ReceiptApi`, `TransactionApi`, `TrustRecordApi`, `PolicyApi`, `RefusalApi`, `AuditApi`), each a thin wrapper over one or more HTTP calls through the shared `Transport`. The public surface includes `execute()`, `createTransaction()` (a second, independent entry point into the identical pipeline via `POST /transactions`), `verify()`, `getLatestVerification()`, `replay()`, `receipt()`, `transaction()` / `transactions()`, `trustRecord()`, `validatePolicy()`, `refusalRecord()` / `verifyRefusalRecord()`, and `verifyAuditEvent()`.

`createBusinessTransaction()` (`typescript/src/builders/createBusinessTransaction.ts`) is the recommended way to build a request body. It takes the fields a caller actually decides (`principalId`, `purpose`, `action`, `target`, `parameters`, `policy`, `signals`, and optional `businessTransactionId`, `authorityType`, `correlationId`, `tenantId`, `sourceSystem`, `submittedBy`) and derives every id-consistency field automatically. `authorityType` defaults to `"SERVICE"`, described in the source as "the correct value for an autonomous agent" (there is no `"AGENT"` value server side, a common first-integration mistake). `businessTransactionId` defaults to a fresh `crypto.randomUUID()`

if omitted, and if you provide your own, you are responsible for its uniqueness, since it doubles as the server's idempotency key.

The Python SDK

`python/parmana/client.py` mirrors the same shape: a `ParmanaClient`, a `ClientConfig` (`python/parmana/config/client_config.py`), and a parallel set of API modules under `python/parmana/api/` (`execution_api.py`, `policy_api.py`, `transaction_api.py`, `trust_record_api.py`, `verification_api.py`, `replay_api.py`, `receipt_api.py`, `refusal_api.py`, `audit_api.py`). `python/parmana/builders.py` is the equivalent of the TypeScript builder. Requires Python 3.10+ (from `pyproject.toml`'s `requires-python`).

The plain HTTP path

Every SDK call ultimately becomes an HTTP request. A minimal integration with no SDK at all needs:

1. A bearer key: `Authorization: Bearer <key>` on every request except `/health`, `/ready`, `/openapi.yaml`, `/openapi.json`, `/api-manifest.json`, `/documentation`, `/reference`, `/refusal/verify`, `/audit/verify`, and the key-discovery routes.
2. A policy that already exists on the target instance, at the exact `(name, version)` you reference, and has a real signed `PolicyChangeApprovalRecord` (see Chapter 7), because production instances enforce this.
3. A well formed `BusinessTransaction` body, with every fact your chosen policy's rules reference present in `signals`, matching either a `boundSignals` entry (derived automatically from `target` / `parameters` server side, but still worth sending consistently) or an `unboundSignalReasons` entry (must be supplied directly).

Concrete example

`examples/tutorials/` has dedicated SDK walkthroughs (grep `examples/tutorials/` for `sdk` in the directory name): the TypeScript and Python quickstart, AI-agent, and production-pattern tutorials each demonstrate a real client construction and a real `execute()` call against a locally spun-up server, no live network needed. `docs/site/guides/typescript-sdk-quickstart.mdx` and `docs/site/guides/python-sdk-quickstart.mdx` cover the same ground for the docs site; their code samples were checked against the actual SDK source above, not copied blind.

How to validate this yourself

- `typescript/src/client/ParmanaClient.ts`, `typescript/src/config/Configuration.ts`, `typescript/src/builders/createBusinessTransaction.ts`, `typescript/test/` for the SDK's own test suite.
- `python/parmana/client.py`, `python/parmana/config/client_config.py`, `python/parmana/builders.py`, `python/tests/`.
- `packages/runtime/src/validators/BusinessTransactionValidator.ts` for the exact cross-consistency checks a hand built transaction must satisfy.
- End to end: agent to Parmana to Paytm for the error catalog of what happens when these checks fail.

Requirements checklist

On the caller side, to make any real call:

- A bearer key, provisioned via `npx tsx scripts/generate-api-key.ts --caller-id "<you>" --credential-holder-type <TYPE>`. `credentialHolderType` must be `USER` if you intend to act as a policy governance checker (Chapter 7); `SERVICE` is typical for an automated integration.
- `allowedPrincipalIds`, if your key is scoped to specific principals, must include whatever `principalId` you submit, or the request is rejected before policy runs. Check with `GET /callers/me`.
- `allowedCapabilities`, if scoped, must include the `action` capability you intend to invoke.

On the server/instance side, verified against `.env.example` and `packages/shared/src/config/Config.ts`:

Variable	Purpose
`PARMANA_POLICY_DIR`	Local policy file directory (used when storage is not Supabase backed).
`PARMANA_KEY_DIR` / `PARMANA_KEY_MATERIAL_JSON`	Local Ed25519 key material (`KEY_PROVIDER=local`).

Variable	Purpose
`PARMANA_STORAGE`	`memory` or `supabase`. `supabase` requires `DATABASE_URL`.
`DATABASE_URL`	Direct Postgres connection string (Supabase or otherwise), used by `pg`, not PostgREST.
`SUPABASE_URL` / `SUPABASE_SERVICE_ROLE_KEY` / `SUPABASE_ANON_KEY`	Only needed for the handful of code paths that still use the Supabase client directly (e.g. CI's read-only policy verification script), not the main request path.
`CRYPTO_MODE`, `PRIMARY_SIGNATURE_PROVIDER`, `SECONDARY_SIGNATURE_PROVIDER`, `HASH_PROVIDER`	Signature/hash algorithm selection (see Chapter 3).
`KEY_PROVIDER`	`local` is the only implemented value as of this writing; anything else throws explicitly (`KeyBootstrap.create()`).
`AWS_REGION`, `AWS_ROLE_ARN`	Required only when using real AWS KMS signing (`KmsSigner.ts`), via Vercel OIDC, no static AWS credentials needed.
`EXECUTION_AUTHORIZATION_TTL_SECONDS`	Default 120.
`PARMANA_API_KEYS`	JSON array of caller entries, the live authentication table.
`RATE_LIMIT_EXECUTE_PER_MINUTE`, `RATE_LIMIT_HEALTH_PER_MINUTE`	Rate limiting (Chapter 16).
`HUBSPOT_PRIVATE_APP_TOKEN`, `GITHUB_APP_ID` / `GITHUB_INSTALLATION_ID` / `GITHUB_APP_PRIVATE_KEY`, `PAYTM_CONNECTOR_URL` / `PAYTM_CONNECTOR_SHARED_SECRET`	Only needed if you intend to exercise the corresponding connector (Chapter 12); a connector is simply not registered if its config is unset, nothing else fails.

Common first-integration mistakes

- Sending `authorityType: "AGENT"`, which does not exist server side; use `"SERVICE"`.
- Hand-building the three id-consistency fields instead of using `createBusinessTransaction()`, and getting one wrong.
- Forgetting that `businessTransactionId` is an idempotency key: resubmitting the same id with different content is rejected as a duplicate before execution, not silently overwritten (see `docs/site/guides/end-to-end-paytm-flow.mdx`).
- Referencing a policy `(name, version)` that exists on disk but has never been approved, on a production instance (Chapter 7). Enforcement is on by default there, so the request is rejected as an ordinary policy REJECT before any rule runs, and an authorization that somehow carries no `policyContentHash` is rejected at the gateway.
- Omitting a `signals` entry for a fact the target policy's rules actually reference; the request is rejected, not silently evaluated with a missing/undefined fact.

Chapter 21: Deployment

What it is

This codebase ships with real configuration for two deployment targets: Vercel (`vercel.json`, serverless Functions) and Fly.io (`fly.toml`, `Dockerfile`, a long-running process). Both run the identical application code; what differs is the execution model, and detail.

Why it was built this way

Two genuinely different deployment shapes were supported deliberately: Vercel for zero-maintenance serverless hosting with automatic scaling, Fly.io for a conventional always-on process with a persistent filesystem and predictable warm state. Neither is presented as strictly better in this codebase; they trade off differently, and the `PARMANA_STORAGE`/repository-selection logic (Chapter 13) is written to work correctly under both.

How it works

Vercel

`vercel.json`:

```
{
  "buildCommand": "npm run openapi && node node_modules/typescript7/bin/tsc -b",
  "installCommand": "npm install --include=dev",
  "rewrites": [{ "source": "/(.*)", "destination": "/api" }],
  "functions": {
    "api/index.ts": {
      "includeFiles": "{openapi/**,policies/**,node_modules/swagger-ui-dist/**,typescript/package.json,
    }
  }
}
```

The build compiles with TypeScript 7, installed as the `typescript7` alias next to TypeScript 6, which lint and documentation tools still need.

Every route is rewritten to a single serverless Function, `api/index.ts`. `includeFiles` is notable: it explicitly bundles `policies/**` into the deployed Function, since the Function's filesystem is otherwise whatever the build produced, nothing more. This is also the exact filesystem that turned out to be **read only** at runtime, which is the deployment-specific

Fly.io

`fly.toml`:

```
app = 'parmana-api'
primary_region = 'bom'

[http_service]
  internal_port = 3000
  auto_stop_machines = false
  min_machines_running = 1

  [[http_service.checks]]
    interval = "30s"
    method = "GET"
    path = "/ready"
```

A conventional container (built from the repo's `Dockerfile`), always running at least one machine, with Fly's own health check hitting `GET /ready` every 30 seconds, the same readiness probe covered in Chapter 19. Unlike Vercel, this filesystem is genuinely persistent and writable for the life of the machine.

A read-only filesystem

`PolicyChangeApprovalService.approve()` (Chapter 7) writes the newly approved policy content via `PolicyRepository.save()`. The original implementation, `FilePolicyRepository`, writes to the local filesystem, this works perfectly on Fly.io (writable, persistent) and in local development, but Vercel's serverless Functions run on a **read only** filesystem outside `/tmp`. The very first real production approval attempt against the deployed Vercel instance failed immediately with `EROFS`. The fix, `SupabasePolicyRepository` (added 2026-09-16, see Chapter 13), writes to a `policies` table in Postgres instead, which works identically on both platforms. This is the clearest concrete example in this codebase of why "which platform am I deploying to" is not a cosmetic choice, it changes which filesystem assumptions are safe to make in application code.

AWS KMS and Vercel OIDC

For real production signing (as opposed to a local Ed25519 key file), this codebase integrates with AWS KMS via Vercel's native OIDC federation, no static, long-lived AWS credentials stored anywhere. `KmsSigner.ts` reads exactly two environment variables directly:

```
AWS_REGION       # e.g. ap-south-1
AWS_ROLE_ARN     # the IAM role Vercel's OIDC token assumes
```

(Confirmed by grepping `packages/crypto/src/providers/signer/KmsSigner.ts` directly; these are the only two `process.env` reads in that file, `KmsSigner requires AWS_REGION to be set` is a real thrown error if `AWS_REGION` is missing.) AWS KMS signing documents the Vercel side of the setup (`vercel env add AWS_REGION production`, and the trust policy connecting Vercel's OIDC issuer to the IAM role).

`KEY_PROVIDER=local` remains the only fully implemented local-key path; setting `KEY_PROVIDER` to anything else (`aws-kms`, `azure-key-vault`, `gcp-kms`, `hsm` were once silently ignored, a real gap closed by making `KeyBootstrap.create()` throw explicitly naming the unimplemented value, see gap 40 in the July session's table). The actual KMS signing path used in production goes through `KmsSigner` / `SignerKeyProviderAdapter` independently of that specific `KEY_PROVIDER` flag, verify the current wiring in `packages/crypto/src/SignerBootstrap.ts` if you need the precise selection logic.

Agent network egress

Parmana checks only the actions that go through it, so an agent host must not be able to reach connector APIs or other systems directly. Enforce this at the network layer, and keep connector credentials off agent hosts. The required checks and the commands to run are in the production guide under "Agent network egress".

Concrete example

There is no example tutorial that actually deploys to either platform (that would require real cloud credentials and is out of scope for a hermetic tutorial suite), but `docs/site/guides/deploy-patterns.mdx` and AWS KMS signing walk through the real, current deployment steps for each target.

How to validate this yourself

- `vercel.json`, `fly.toml`, `Dockerfile` (repo root) for the two real platform configs.
- `packages/crypto/src/providers/signer/KmsSigner.ts` for the exact AWS env vars actually read.
- `the policies table migration` for the schema the fix depends on.
- AWS KMS signing for the setup steps.

Requirements

Vercel deployment:

- `AWS_REGION`, `AWS_ROLE_ARN` (for KMS signing) or local key material via `PARMANA_KEY_MATERIAL_JSON` (for local Ed25519 signing, not recommended for production but functional).
- `PARMANA_STORAGE=supabase` with `DATABASE_URL` set, since the filesystem cannot be relied on for anything written at runtime, including approved policy content.
- Every other env var from Chapter 20's requirements table.

Fly.io deployment:

- The same core env vars, `PARMANA_STORAGE=memory` is viable here if you genuinely want no external database (the filesystem is real and persistent), but `supabase` remains the production-realistic choice for durability across machine restarts.
- `fly.toml`'s health check expects `GET /ready` to respond within its configured timeout (5s); see Chapter 19 for what that route actually checks.

Chapter 22: Testing Philosophy

What it is

Two disciplines run through this codebase's tests: hermetic-first testing (no real network, no real database, no real cloud credentials required to run the suite), and treating the runnable tutorial suite under `examples/tutorials/` as a first-class form of documentation, not a separate, optional thing bolted on after the real code and real tests exist.

Why it was built this way

A test that needs a live Supabase connection or real AWS credentials to run cannot run in CI without secrets, cannot run offline, and is slow. A prose document describing behavior can silently go stale the moment the underlying code changes, since nothing forces it to be re-checked. Both problems have the same root cause, a gap between what is claimed and what is actually exercised, and this codebase addresses both with the same tool: code that asserts real behavior and fails loudly the moment that behavior regresses.

How it works

Hermetic-first testing

Rather than mocking at the framework level, unit tests in this codebase commonly construct a minimal fake object implementing just the one method actually called. `packages/storage/tests/unit/` has several real examples (`business-transaction-repository-duplicate-consistency.test.ts`, `postgres-rate-limit-store.test.ts`, `supabase-business-transaction-repository.test.ts`, among others) that build a `FakePool`-shaped object standing in for `pg.Pool`, exposing only `.query(sql, params)` against an in-memory data structure, no real network socket ever opens. This same pattern is used in the example tutorials that exercise real Postgres-backed classes hermetically (`examples/tutorials/115-per-limiter-rate-limit-stores`, `116-supabase-policy-repository`, `117-maker-checker-one-shot-scripts`, each fakes just enough of `pg.Pool` for the class under test to function correctly, verified by reading their `run.ts` files directly).

`vitest.config.ts` (repo root) sets `NODE_ENV=test` implicitly for the whole suite via Vitest's own default behavior, and `packages/storage/src/StorageFactory.createFromEnvironment()` reads that explicitly, returning `MemoryStorageProvider` unconditionally under `NODE_ENV=test`, regardless of what `PARMANA_STORAGE` happens to be set to in the ambient environment. This exists specifically so importing application code during test collection never accidentally constructs

The tutorial suite as documentation

`examples/tutorials/` currently runs to 117 numbered tutorials plus `examples/scenarios/`, invoked together via `npm run examples` (`scripts/run-examples.ts`). Each tutorial is a standalone, runnable script that exercises real production code (not a simplified re-implementation) against a hermetic environment, either an in-process Express server on an ephemeral port, or a fake client standing in for one external dependency, and prints its own pass/fail assertion at the end. The reasoning, stated directly in this codebase's own review of itself: a runnable tutorial is falsifiable in a way prose is not, if the underlying behavior changes, the tutorial's own final assertion fails the next time anyone runs it, where a paragraph of documentation making the same claim would simply keep reading as true.

One entry is deliberately excluded from the automated `npm run examples` run: `examples/tutorials/09-rest-api`, which POSTs to a live server the runner does not start. A comment in `scripts/run-examples.ts` explains why and points at its own `README.md` for how to run it individually. `examples/04-verified-execution` used to be excluded too, because it bound a fixed port; it stopped running unnoticed, and now listens on a free port and runs with the rest.

Attack scenarios

`npm run evaluate` (`scripts/evaluate.ts`) groups existing tests into 16 attack scenarios, listed in `evaluations/scenarios.json`: replay, forgery, acting without a signed approval, policy substitution, gateway bypass, record tampering and more. Each scenario names what the attacker controls, the invariant and the test files that check it. The runner runs those files once and reports each scenario `BLOCKED`, `FAILED` or `NOT RUN`, with an `evaluation-report.json` tied to the commit. It adds no new tests: it gives an evaluator one result per attack

instead of 3,000 test names. `tests/architecture/evaluation-scenarios.test.ts` fails if a scenario points at a test file that no longer exists or a claim number missing from.

Fuzzing

Property based tests with fast-check send damaged and arbitrary input where the server reads what others wrote: request bodies (`POST /execute` and `POST /transactions` , a valid transaction with fields removed or replaced by arbitrary JSON, and arbitrary JSON), policy files (validation must accept or refuse with `PolicyValidationError` , and a policy that validates must evaluate any signals to `APPROVE` or `REJECT`), the offline verifier (any input is `valid: false` with a reason) and approval parsing. They run with every `npm test` . `authorization` gave a bare `500` , the policy validator threw `TypeError` on fields of the wrong type, and the offline verifier threw on malformed hybrid signatures. All three are fixed, each with an exact test.

Mutation testing

A count of tests, or line coverage, says little about whether tests would notice a wrong answer. `npm run mutation -- <package>` (`scripts/mutation.ts` , Stryker) measures that instead: it makes small deliberate changes to a package's source, such as turning `<=` into `<` or forcing a condition to `true` , one at a time, and runs the package's own tests against each. The share of changes the tests catch is the mutation score. It is slow, because every change runs the package's whole test suite, so it is run by hand rather than in CI.

77.7% for `approval` , 50.3% for `policy` , 67.6% for `execution-gateway` and 65.4% for `crypto` . The surviving changes pointed at real gaps: approval scope comparators other than `eq` and `lte` , what rule operators do with a value of the wrong type, gateway options that could be dropped on the way to the envelope verifier, session binding fields never tested alone, and the offline verifier's hybrid path. Two led to behavior changes: a request whose signals do not have the policy's declared types is now refused before any rule runs, and an Execution with only some of Each gap that matters for security now has a test, checked by putting the targeted change back by hand.

The crypto score was measured on Node 22, where the ML-DSA-65 tests skip themselves, so it undercounted. `npm run mutation` now refuses an older Node than 24, the version CI uses. Re-run on Node 24 after the fixes: `envelope-verifier` 94.0%, `approval` 93.9%, `policy` 96.0%, `crypto` 85.0%, `execution-gateway` 83.6%. What survives is almost all equivalent (a guard a later check repeats), advisory (the rule overlap warnings) or message wording. and the tutorial `README.md` files cite real file paths as evidence for their claims. This codebase treats a stale citation as a real defect worth finding and fixing, not a cosmetic nit, a citation nobody automatically checks will eventually go stale, and a claim whose evidence has silently rotted is functionally the same as an unverified claim. is explicit that a citation should be understood in the context it was written (a citation may be correct as of the date it was added even if later superseded, in which case the surrounding prose says so rather than the citation being silently rewritten to point somewhere else, doing so would falsify the historical record rather than fix a real problem).

CI

`.github/workflows/ci.yml` runs, on every push and pull request: install (`npm ci`), build (`npm run build`), lint (`npm run lint`), typecheck (`npm run typecheck`), a guard against retired terminology (a grep-based check, not a type or lint rule), the full test suite (`npm test`), and, in a separate job, `scripts/verify-policy-changes-approved.ts` against whichever `policies/**/policy.json` files changed in that push or PR, this is the fail-closed governance gate described in Chapter 7, real and enforced in CI today, not yet wired as a GitHub required status check due to a plan/visibility limitation external to this codebase (see §2.26's "Preventive Git-layer enforcement" entry for the exact constraint).

The local pre-commit hook (this repo's own `.git` hooks, not GitHub's CI) additionally runs the full example suite (`npm run examples`) before allowing a commit, meaning every commit landed on `main` has, at minimum, had all currently-registered tutorials pass on the committing machine at commit time.

Concrete example

Any tutorial run demonstrates the philosophy directly:

```
npx tsx examples/tutorials/117-maker-checker-one-shot-scripts/run.ts
```

This tutorial spins up a real Express app on an ephemeral local port, exercises the real maker-checker HTTP endpoints and the real operational scripts used to approve production policies, and prints a final `✓` / `✗` verdict,

entirely hermetically, no network access, no real credentials.

How to validate this yourself

- `packages/storage/tests/unit/` for the fake-`pg.Pool` unit test pattern.
- `scripts/run-examples.ts` for the full registered tutorial list and the two documented exclusions.
- `.github/workflows/ci.yml` for the exact CI gates. -'s own header/preface section for the citation-integrity discipline stated in its own words.
- Any individual `examples/tutorials/*/README.md`, each states what it proves and why it matters, in the same structure this book uses.

Requirements

None beyond what's already needed to run the codebase locally (Node.js, `npm install`). No tutorial or unit test in the default `npm test` / `npm run examples` runs requires real AWS, Supabase, or connector credentials; that is the entire point of the hermetic-first discipline described above.